作品名称：FPGA 协同 CNN 辅助车牌识别

（不要出现任何涉及学校名称等内容）

摘要

800 字内

本作品以车牌图像采集与识别作为主要导向，基于 **PYNQ-Z2** 设计了一套 FPGA 硬件加速与 CNN 深度学习模型协同工作的视觉系统。针对传统车载和安防视觉将全图输入车牌识别导致的高延迟与算力浪费等痛点，本系统实现了从视频采集、硬件图像前处理与候选区域筛选、事件触发到上层 AI 识别的全流程闭环。

在**底层硬件加速设计**方面，系统通过 USB 摄像头采集图像，把 640×480 灰度帧送入 FPGA 流水线。然后在底层通过 3×3 **Sobel 边缘提取**、网格边缘统计和四连通区域分析来提取结构信息丰富的候选 ROI，输出区域坐标和二值边缘图。系统还设计了**双 AXI DMA 架构**：主通道仅回传 12 字节的极简 ROI 坐标，而辅通道则独立回传真实边缘图，在剔除大部分以上无效背景的同时，确保了硬件底层数据对前端可视化的相对透明，兼顾结果输出和硬件验证。

在**上位机与系统架构**方面，本系统采用 C/S 网络架构。客户端基于 **PyQt5** 与 **TCP 协议**构建监控台，具备跨设备的远程交互能力。通过多线程设计实现了视频流与指令流的分离，并支持双画面实时监控。抓拍产生的轻量化彩色图像，通过**异步 HTTP 请求**传输到电脑端，由基于 **ONNX Runtime** 部署的**开源 CNN 模型**完成车牌定位与字符识别。

在**应用功能**方面，系统不仅支持手动抓拍，还在 PYNQ 的 PS 端引入了**时空双防抖机制**（即基于持续时长与空间一致性$IoU \geq 0.6$判定）。仅在目标稳定且有效时，才触发原图 ROI 裁剪并调用 CNN 识别，配合冷却机制极大降低后端的无效算力消耗与存储冗余。

在**工程验证与性能评估**方面，本作品遵循较为严谨的 V 模型开发流程。首先构建了位精确的 **Python 黄金参考模型**，完成了从 RTL 仿真、双 DMA 综合布局布线到时序收敛的严格验证。系统还内置了按需触发的 **ARM OpenCV 同帧对**

比，通过直观的数据量化了 FPGA 硬件流水线相较于纯软件算法的确定性加速优势。

本作品不仅是一个车牌识别原型，更为嵌入式视觉前端与智能识别的组合应用提供了极具参考价值的工程例子，展示 FPGA 流式图像前处理与软件识别服务协同工作的实现。

关键词 PYNQ-Z2 双 AXI DMA 动态 ROI 筛选 PyQt5 CNN 模型 Sobel 检测

目录

第一部分　作品概述 4
1.1　功能与特性 4
1.2　应用领域 4
1.3　主要技术特点 5
1.4　主要性能指标 6
1.5　主要创新点 7
1.6　设计流程 7
第二部分　系统组成及功能说明 8
2.1　整体介绍 8
2.2　硬件系统介绍 13
2.2.1 硬件整体介绍 13
2.2.2 机械设计介绍 13
2.2.3 电路各模块介绍 14
2.3　软件系统介绍 48
2.3.1 软件整体介绍 48
2.3.2 软件各模块介绍 49
第三部分　完成情况及性能参数 69
3.1　整体介绍 69
3.2　工程成果 71
3.3　特性成果 72
第四部分　总结 77
4.1　可扩展之处 77
4.2　心得体会 77

第一部分　作品概述

1.1 功能与特性

400 字内（可加重要图）；

①实现**双画面展示**：该作品可以同步在左边显示摄像头拍摄到的原图和 ROI 框选后的图，并在右边展示 FPGA 处理后的二值边缘图像，以供对比展示；

②**手动与自动抓拍**：手动抓拍时可以一键抓取并保存原始彩色 ROI 保存在 jupyter 文件夹根目录下；自动抓拍则会在观测区域稳定后执行，且只会在识别到车牌并抓怕成功后上传到根目录，其余时间段即使识别到 ROI 区域也不会抓拍，避免无效拍摄。同时通过**冷却机制**减少重复抓拍；

③车牌识别和结果回传：将抓拍的 ROI 局部图像传输至电脑端，由客户端调用 **CNN 模型**完成车牌识别，并实时回传识别的车牌号与推理耗时；

④性能对比和监测：可以在界面下方显示 FPGA 调用耗时，候选区域**面积缩减率**等关键指标，同时可以和**同帧 ARM OpenCV** 进行对比耗时，展示 FPGA 处理的优势；

⑤远程交互和控制：只需要通过 **PyQt5** 客户端输入 IP 地址连接 PYNQ-Z2，就可以进行视频查看、抓拍等操作，实现跨设备的远程交互控制；

1.2 应用领域

400 字内（可加重要图）；

本作品面向 USB 摄像头的车牌采集与识别原型验证，可用于停车场、园区出入口等场景的实验演示，实现候选区域观察、事件触发抓拍与车牌信息读取。

我们的硬件前端主要依据 sobel 算法 + ROI 筛选生成候选区域，与具体字符识别模型关联不大。因此可以通过更换后端 CNN 深度学习模型，进一步探索例如铭牌或者文字标签这样结构信息比较丰富而且环境相对简单的采集应用。

此外，本系统完整呈现了图像算法硬件化、PS-PL 异构协同及网络交互的工程范式。现阶段，作品聚焦于背景简单、目标清晰的受控场景进行高可靠验

证，不仅可作为智能识别的边缘原型，也可作为 FPGA 硬件加速与软硬协同算法的直观教学。现阶段主要针对背景相对简单、目标清晰的受控场景开展验证。



1.3 主要技术特点

400 字内；

本项目车牌识别系统的主要技术特点有：

1.流式硬件处理

采用 RTL 来实现 Sobel 运算，通过行缓存构建 3×3 局部像素窗口，用流水线方式完成边缘提取；

2.网格化区域分析

将图像划分为 8×6 个网格，根据边缘点数量生成活跃网格，再通过四连通分析确定最大连通区域及其包围框，缩小区域分析规模；

3.双通道 DMA 数据回传

设计主从双路回传机制，分别回传 ROI 六字段和完整的二值边缘图，为原图裁剪与结果验证提供真实的数据；

4.上位机的灵活控制和友好显示界面

基于 PyQt5 的跨平台上位机监控终端，用户可以通过上位机发送相关按键功能来实现对车牌的识别和自动抓拍。同时“视控分离”：视频流通过 TCP 协议低延迟回传保障界面流畅；触发抓拍后，图像通过 HTTP 请求至上位机后端的 CNN 服务，两者互不干扰。

5. 功能的多样性

本系统不仅可以再在上位机看到画面实时效果，也可实现手动抓拍、自动抓拍、AI 识别、同帧 Open CV 处理对比等功能。

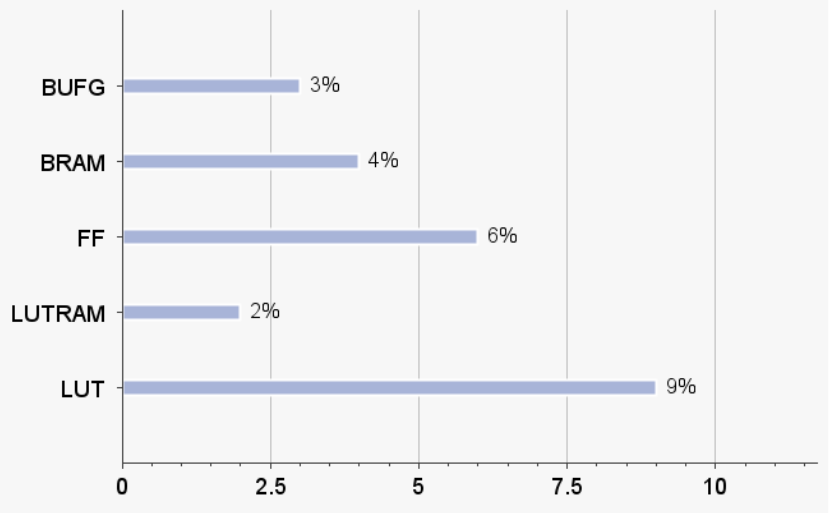
1.4 主要性能指标

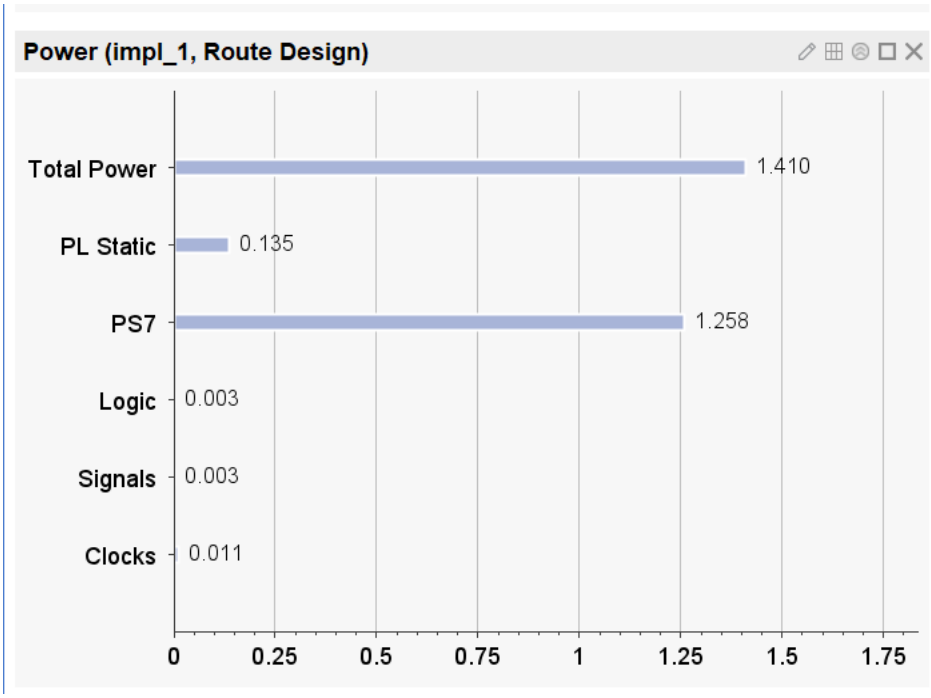
200 字内（建议用表格）；

Timing (impl_1, Route Design)

Report	WNS	WHS
impl_1, Timing Summary - Route Design	10.360	0.020

Utilization (impl_1, Place Design)





1.5 主要创新点

200 字以内（逐点给出）；

1. 纯硬件提取网格统计的候选区域

传统方案一般把将连通域分析交由软件处理，本系统在 FPGA 纯硬件层手写 RTL 状态机，利用 48-bit 位图扩张法实现四连通区域合并与最大 ROI 筛选，处理时间得到较大缩短；

2. 时空双防抖（IoU + 时间）事件触发机制

在 PS 端设计自动抓拍逻辑，系统不盲目处理每一帧，而是结合“持续稳定时长”与“空间交并比（$IoU \geq 0.6$）”双重判定。只有在目标稳定停留且有效时，自动模式下才触发抓拍，降低后端的无效调用率。

3. 视控分离的现代上位机架构

基于 **PyQt5** 构建桌面客户端，采用 **C/S 网络架构**实现软硬件相对独立。通过多线程设计将 TCP 视频流接收、异步 HTTP 识别请求等进行隔离，避免网络延迟导致的界面卡顿。终端集成了双画面同帧对比、手动与自动抓拍及识别结果显示等功能，完整展现软硬协同的处理过程。

1.6 设计流程

200 字内（可加重要图）；

算法定义与参考模型建立(2 周)：明确图像规格、算法规则与数据接口，划定 640×480 灰度输入与双 DMA 数据流格式。完成顶层模块划分，并优先建立一套精确的 Python 软件黄金参考，为后续硬件验证确立基准。

RTL 模块开发与仿真验证(3 周)：并行开发 Sobel 边缘提取、8×6 网格统计及四连通 ROI 分析模块，通过编写 Testbench 进行功能仿真与像素级比对，逐模块完成测试用例覆盖，确保与软件参考结果严格对齐。

总线集成与时序收敛(2 周)：基于 AXI4 总线集成双 DMA 分流架构，完成 Block Design 搭建。解决跨模块握手与反压冲突问题，进行严格的综合实现与时序检查，攻克时序问题并生成 Overlay。

上位机开发与板端联调(2 周)：将系统部署至 PYNQ-Z2 开展板端图像验证，接入 USB 摄像头并搭建 TCP 视频服务。基于 PyQt5 开发客户端，实现双画面实时显示与通信交互，优化数据传输延迟。

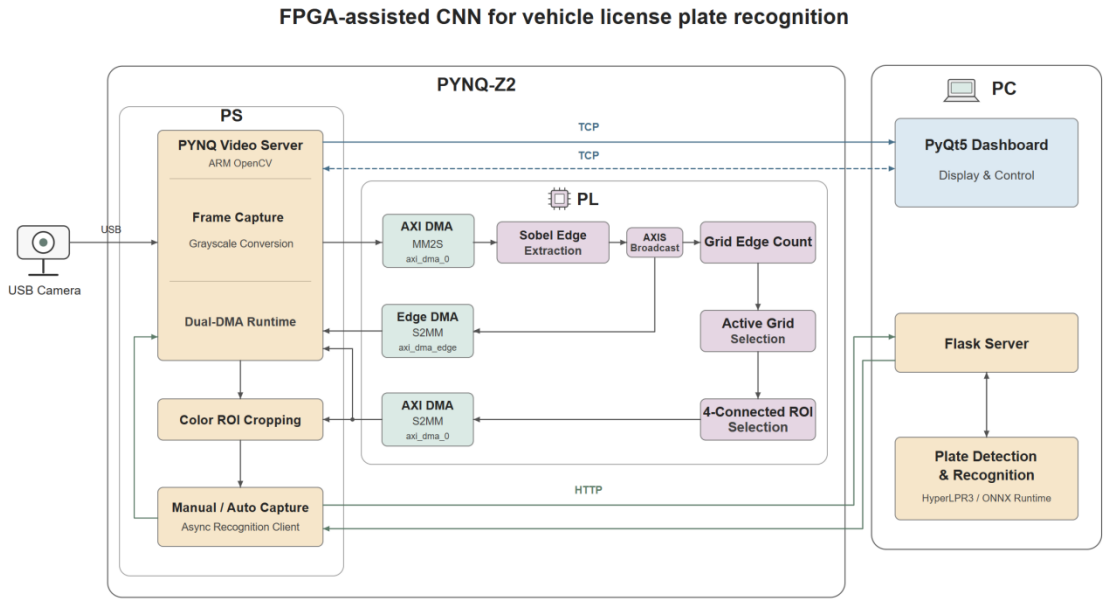
系统测试与文档编写(2 周)：加入时空双防抖自动抓拍机制，接入电脑端 CNN 模型完成异步识别。开展性能测试与长时稳定性验证，记录端到端耗时、缩减率等关键指标，同时编写技术文档,完成项目交付准备。

第二部分　系统组成及功能说明

阐述具体的设计细节（图文结合）

2.1 整体介绍

给出系统整体框图，各子模块标注清楚，并进行整体的文字说明，需要表达出各模块之间的关系。



本系统采用“PYNQ-Z2 异构平台＋PC 端上位机”的协同架构，以 USB 摄像头作为图像输入，完成视频采集、FPGA 前处理、候选 ROI 筛选、抓拍及车牌识别。PYNQ-Z2 内部由 PS 与 PL 协同工作。PS 端运行 PYNQ Video Server，负责图像采集、灰度转换、DMA 调用、彩色 ROI 裁剪及抓拍管理；PL 端通过 Sobel 边缘检测、网格边缘统计和四连通区域分析，生成候选 ROI，并回传真实二值边缘图。

PC 端包含 PyQt5 Dashboard、Flask Server 以及车牌检测与识别模型。Dashboard 负责视频展示和操作控制；Flask 服务接收候选彩色图像，调用 HyperLPR3 模型完成车牌识别，将结果返回板端并更新至监控界面。

2.1.1 FPGA 平台内部模块详解

PYNQ-Z2 内部模块可以分为 PS 端软件模块与 PL 端硬件模块。

一、PS 端模块

PYNQ Video Server（板端视频服务）：作为 PS 端的主控程序，组织图像采集、硬件处理、网络发送和上位机指令执行。图中的 ARM OpenCV 用于图像处理，并在用户触发时执行软件参考算法，和同帧 FPGA 处理结果对比。其中 Frame Capture 负责从 USB 摄像头读取彩色帧；Grayscale Conversion 把彩色

帧转换成 640×480、8 位灰度图以供 PL 端处理，同时会保留原始彩色图像用于显示裁剪。

Color ROI Cropping：根据 FPGA 返回的区域坐标，从对应的原始彩色帧中裁剪候选区域。硬件使用灰度图进行前处理，裁剪模块则保留候选区域的彩色信息，为后端 CNN 检测与识别提供输入。

Manual/Auto Capture（手动与自动抓拍）：手动模式响应上位机抓拍指令，保存当前有效 ROI 并提交识别。自动模式根据 ROI 持续时间与空间一致性判断稳定状态，满足条件后提交候选图像，识别通过后保存，并进入冷却阶段。同一模块中的 Async Recognition Client 则是负责异步 HTTP 通信，在后台发送候选图像、接收识别结果并更新状态。

二、PL 端模块

AXI DMA MM2S / S2MM（主 DMA）：图中上方的 MM2S 与下方的 S2MM 属于同一个 axi_dma_0，分别表示两个传输方向。MM2S 将 PS 准备的灰度帧转换为 AXI4-Stream 数据送到 Sobel 模块；S2MM 接收 ROI 模块输出的六字段结果，返回 PS 端。

Sobel Edge Extraction：通过行缓存与窗口寄存器构建 3×3 像素窗口，计算图像梯度，经过幅值处理和阈值二值化后输出边缘像素流。

AXIS Broadcast：将 Sobel 输出分为两条支路：一条进入 Grid Edge Count 进行区域分析，另一条进入 Edge DMA 回传边缘图。模块通过握手信号管理两路接收。

Grid Edge Count：将图像划分为 8×6 个网格，每格 80×80 像素，分别累计网格中的边缘点数量。统计结果用于描述图像不同区域的结构密集程度。

Active Grid Selection（活跃网格选择）：根据网格计数与活跃阈值进行比较，生成活跃网格位图，作为四连通区域分析的输入。该功能在当前工程中由 ROI 模块内部实现。

4-Connected ROI Selection（四连通 ROI 筛选）：通过硬件状态机分析活跃网格之间的上下左右相邻关系，选择网格数量最多的连通区域，计算其包围框。输出内容包括区域位置、宽高、最大连通域网格数及面积缩减率。

Edge DMA S2MM（边缘图 DMA）： 由 axi_dma_edge 实现，接收 **AXIS Broadcast** 输出的完整二值边缘图并返回 PS 端。该通路为 Dashboard 提供 FPGA 实际生成的边缘图用于展示。

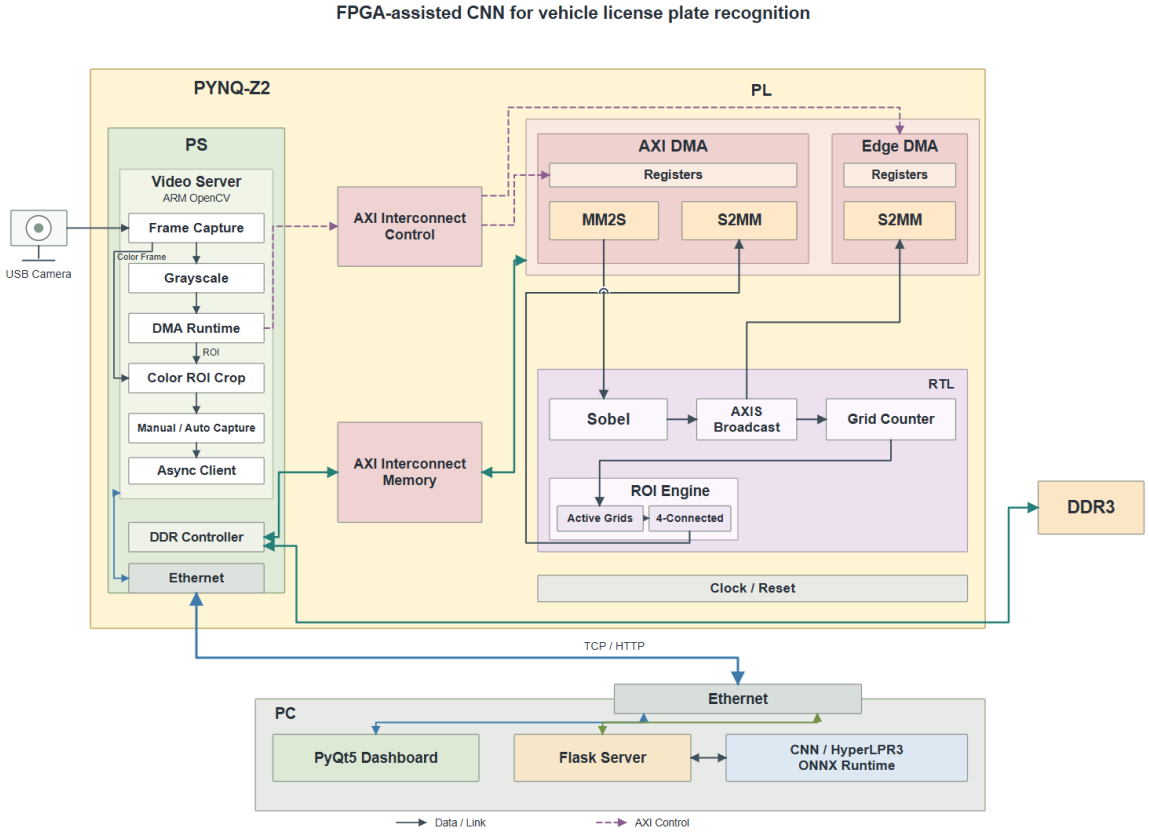
2.1.2 协处理器（PC / 上位机）内部模块

PyQt5 Dashboard： 接收 PYNQ Video Server 发送的视频和状态数据，完成双画面显示：左侧展示彩色原图、ROI 框及局部图，右侧展示 FPGA 二值边缘图。界面同时显示处理耗时、候选面积缩减率、抓拍状态和识别结果。用户通过 Dashboard 发送手动抓拍、自动抓拍开关及 ARM 对比指令。

Flask Server（识别服务接口）： 接收板端 Async Recognition Client 发送的候选彩色图像，完成图像解码、模型调用和结果封装，并通过 HTTP 响应返回 JSON 格式的识别结果。

Plate Detection & Recognition： 通过 ONNX Runtime 执行推理，候选彩色图完成车牌检测和字符识别。FPGA 提供结构信息丰富的 ROI 候选区域，该模块进一步判断其中是否存在车牌并输出车牌号。

2.1.3 具体的模块间关系：



1. 控制流

用户在 PyQt5 Dashboard 中执行操作，控制指令通过 TCP 连接发送至 PYNQ 的 Video Server 模块。板端主程序接收指令后，分别执行手动抓拍、自动模式切换或 ARM 软件对比。

视频处理过程中，PYNQ Video Server 调用 DMA Runtime，由 PS 经 GPIO 和 AXI 控制互连访问两个 DMA 的 AXI-Lite 寄存器。Runtime 先启动 ROI 和边缘图两路接收，再启动灰度帧发送，等待本帧传输完成后读取结果，供后续裁剪显示。

自动模式下，抓拍模块依据 ROI 稳定状态发起识别请求。Async Client 将请求交给后台线程处理，Flask 服务通过 HTTP 响应返回识别结果；PS 端更新识别及自动保存状态，再由 PYNQ Video Server 通过 TCP 更新 Dashboard。

2. 数据流

图像输入路径：USB 摄像头输出彩色帧，经 Frame Capture 读取后，由 Grayscale 生成灰度图，再通过 AXI DMA 的 MM2S 通道送入 Sobel 完成后续一系列底层处理。

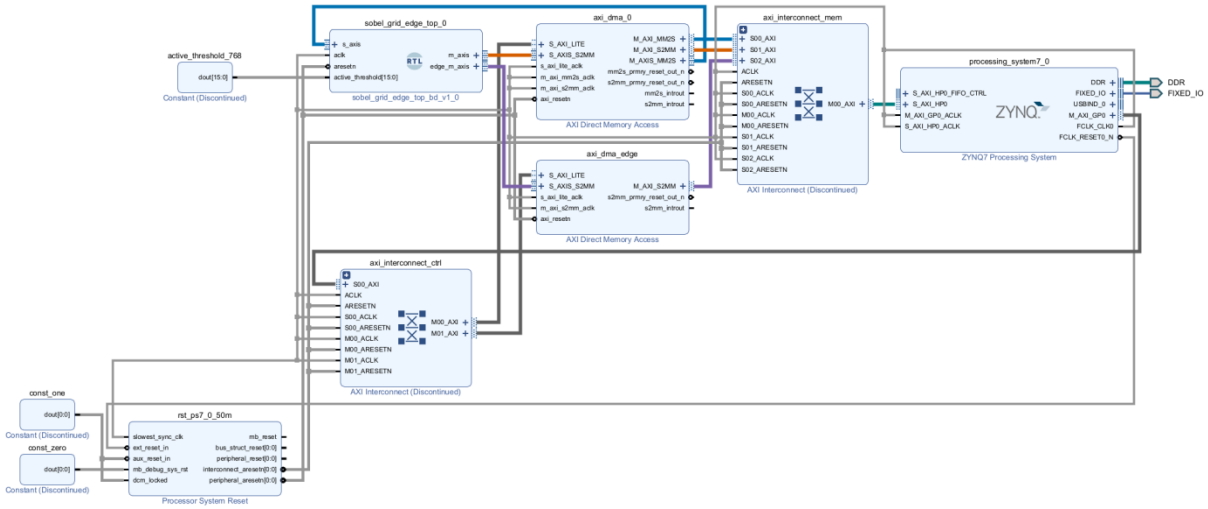
ROI 计算路径：Sobel 输出经过 AXIS Broadcast 进入 Grid Edge Count，依次完成网格统计、活跃网格选择和四连通 ROI 筛选。最终六字段结果通过主 DMA 的 S2MM 通道返回，供 Color ROI Cropping 裁剪对应彩色区域。

边缘图回传路径：AXIS Broadcast 的另一条支路将完整二值边缘图送入 Edge DMA S2MM，返回 Dual-DMA Runtime。板端将原图、边缘图及对应区域信息通过 TCP 发送至 Dashboard，完成双画面展示。

车牌识别路径：候选彩色图由 Manual/Auto Capture 模块提交，经 Async Recognition Client 发送至 Flask Server，再交给 HyperLPR3 / ONNX Runtime 完成检测与识别。结果沿 HTTP 响应返回 PS 端，随后通过 TCP 状态消息更新至 PyQt5 Dashboard。

2.2 硬件系统介绍

2.2.1 硬件整体介绍；

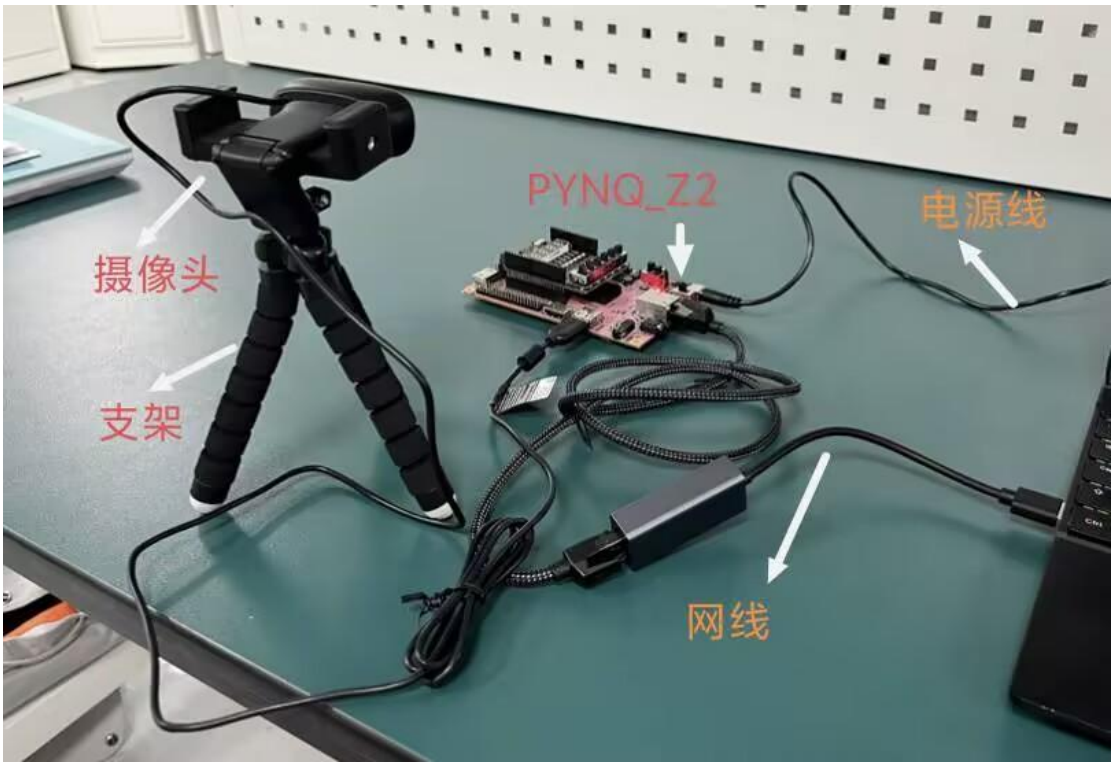


详细的控制流和数据流可参考上文整体介绍部分。

2.2.2 机械设计介绍

（如果有的话，从总体到局部，逐级给出各组件的具体设计图，可以是 CAD 文件截图或者手绘图片）；

系统采用 PYNQ-Z2 开发板、USB 摄像头与笔记本电脑搭建桌面实验平台，没有设计专用的机械结构。实验中通过固定摄像头位置，保持采集条件相对稳定。



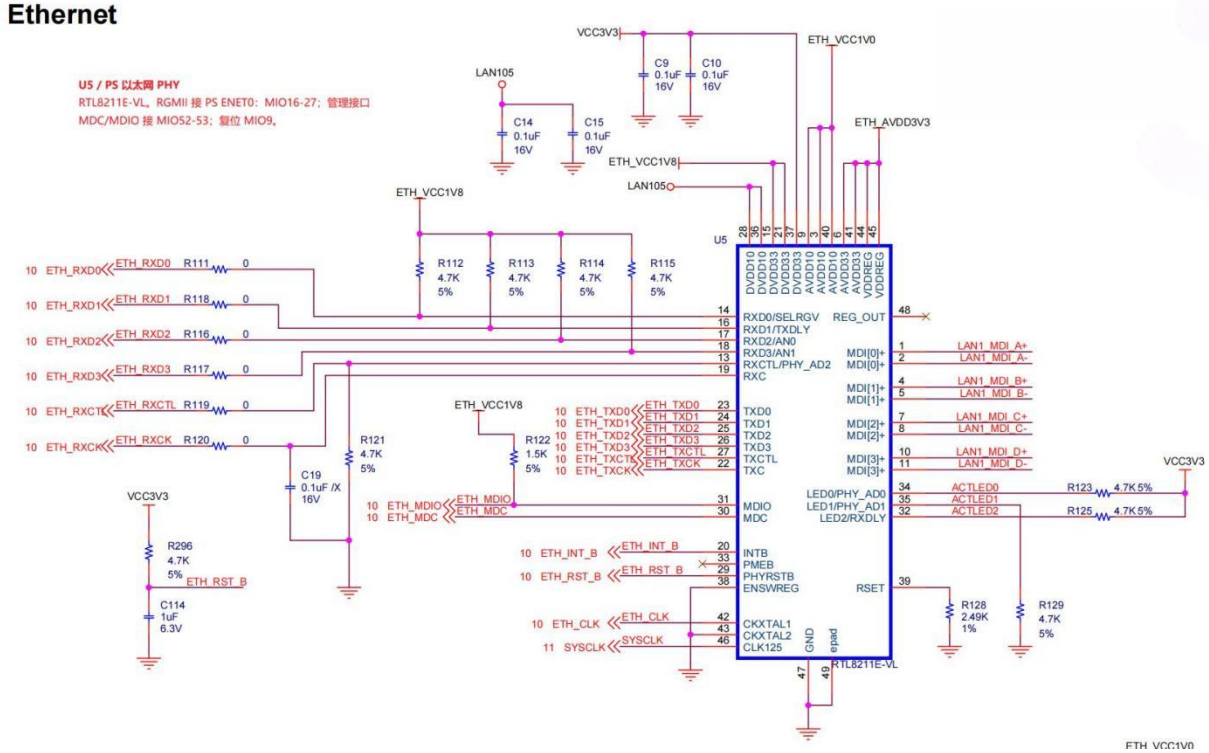
2.2.3 电路各模块介绍

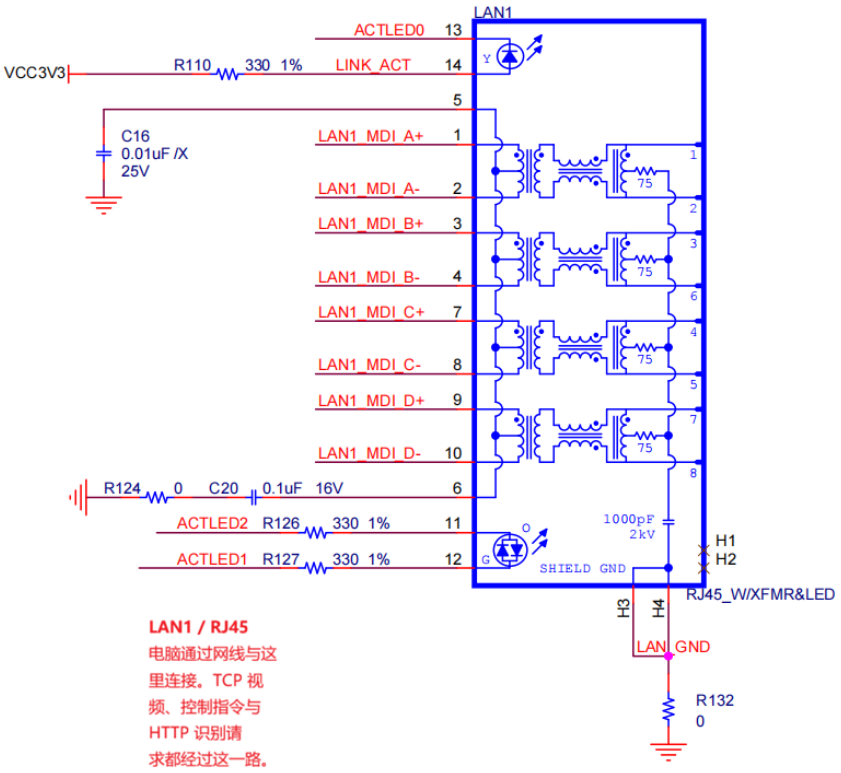
（从总体到局部，逐级给出各模块的具体设计图，并标记出关键的输入、输出信号线，可以是电路图、SCH 原理图、PCB 版图等截图）；

本系统选用 PYNQ-Z2 开发板作为硬件平台，以板载 Zynq-7000 系列 XC7Z020 芯片为核心，利用处理系统 PS 与可编程逻辑 PL 协同完成图像采集、边缘检测和动态感兴趣区域筛选。系统主要使用 USB 摄像头接口电路、RTL8211E-VL 以太网接口电路、DDR3 存储电路、microSD 启动与存储电路，以及 Zynq 主控、时钟、复位和电源电路。

其中，USB 接口用于连接摄像头，获取待处理图像；DDR3 用于存放输入图像、边缘图及处理结果；以太网接口用于开发板与电脑之间的图像传输、控制交互和识别结返回；microSD 用于系统启动、程序及文件存储。PS 端负责图像采集、预处理与任务调度，PL 端实现 Sobel 边缘检测、网格边缘统计和四连通 ROI 筛选，两者通过 AXI 总线及 DMA 完成数据交换。各部分电路图如下：

Ethernet

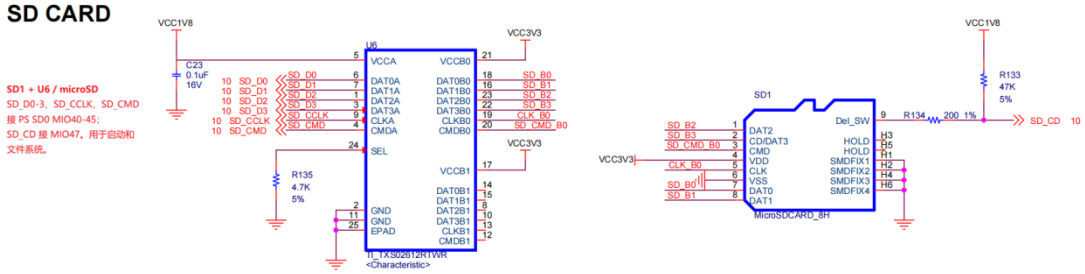

U5 / PS 以太网 PHY
RTL8211E-VL，RGMII 接 PS ENET0；MIO16-27；管理接口
MDC/MDIO 接 MIO52-53；复位 MIO9。
VCC3V3
ETH_VCC1V0
ETH_AVDD3V3
ETH_VCC1V8
LAN105
C9 0.1uF 16V
C10 0.1uF 16V
C14 0.1uF 16V
C15 0.1uF 16V
R112 4.7K 5%
R113 4.7K 5%
R114 4.7K 5%
R115 4.7K 5%
U5
RTL8211E-VL
RXD0/SELRGV
RXD1/TXDLY
RXD2/AN0
RXD3/AN1
RXCTL/PHY_AD2
RXC
TXD0
TXD1
TXD2
TXD3
TXCTL
TXC
MDIO
MDC
INTB
PMEB
PHYRSTB
ENSWREG
CKXTAL1
CKXTAL2
CLK125
REG_OUT
RSET
GND
epad
ETH_RXD0
ETH_RXD1
ETH_RXD2
ETH_RXD3
ETH_RXCTL
ETH_RXCK
ETH_TXD0
ETH_TXD1
ETH_TXD2
ETH_TXD3
ETH_TXCTL
ETH_TXCK
ETH_MDIO
ETH_MDC
ETH_INT_B
ETH_RST_B
ETH_CLK
SYSCLK
R111 0
R118 0
R116 0
R117 0
R119 0
R120 0
R121 4.7K 5%
R122 1.5K 5%
C19 0.1uF /X 16V
R296 4.7K 5%
C114 1uF 6.3V
R123 4.7K 5%
R125 4.7K 5%
R128 2.49K 1%
R129 4.7K 5%
LAN1_MDI_A+
LAN1_MDI_A-
LAN1_MDI_B+
LAN1_MDI_B-
LAN1_MDI_C+
LAN1_MDI_C-
LAN1_MDI_D+
LAN1_MDI_D-
ACTLED0
ACTLED1
ACTLED2

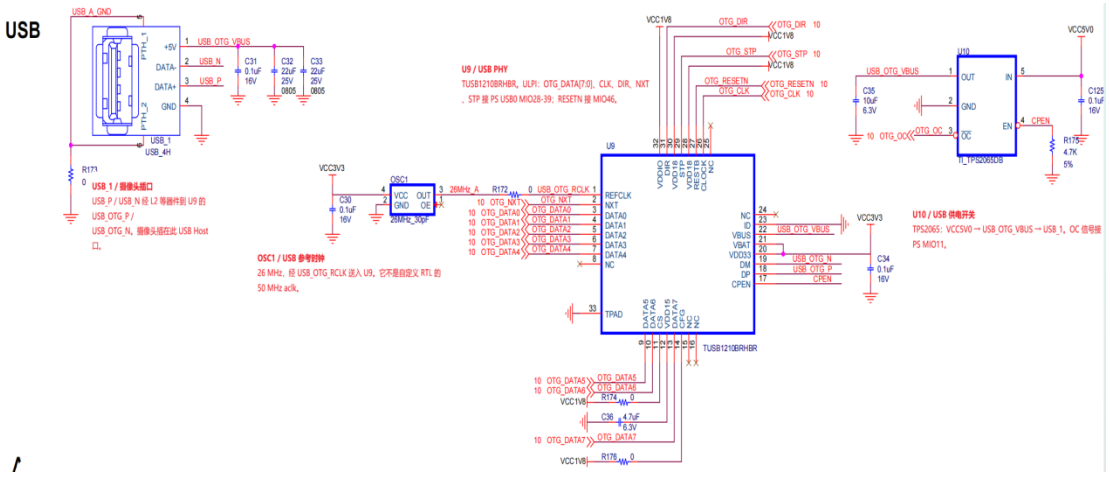


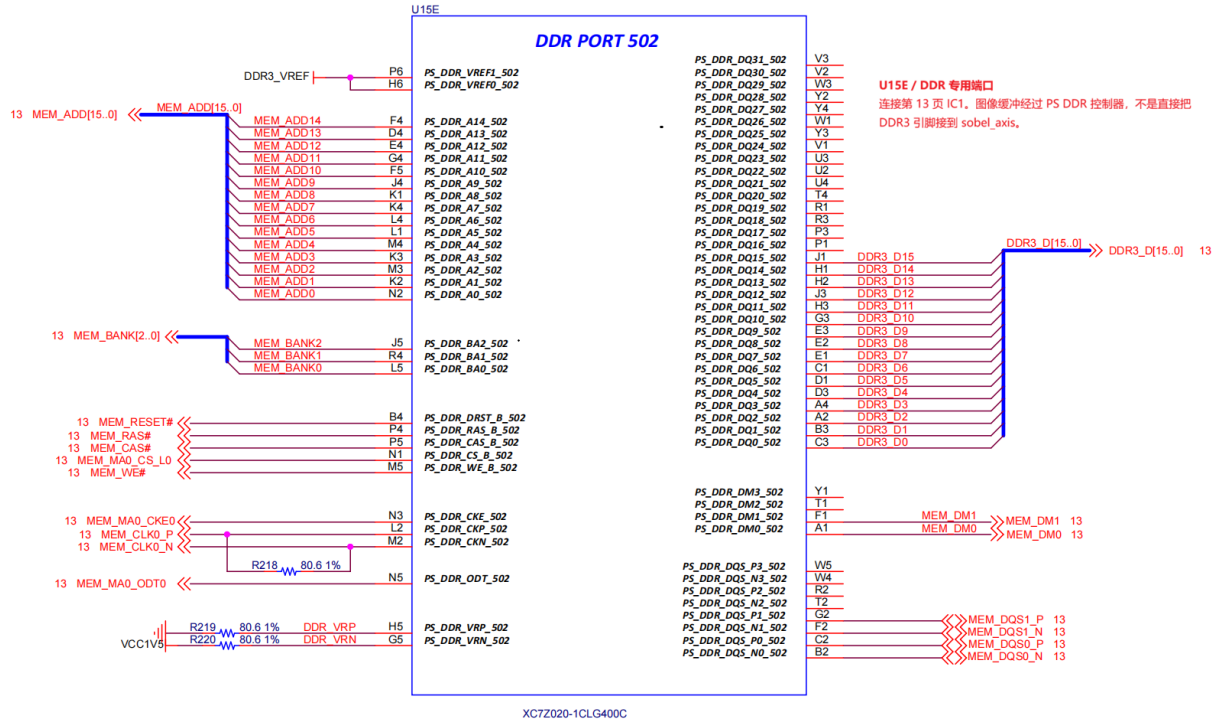
LAN1 / RJ45

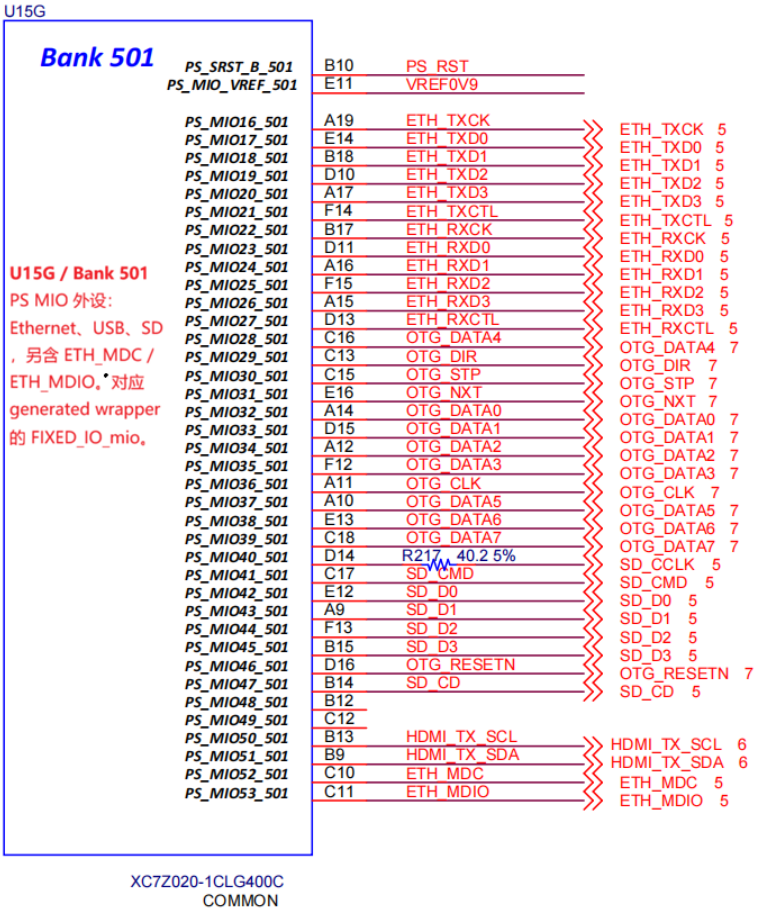
电脑通过网线与这里连接。TCP 视频、控制指令与 HTTP 识别请求都经过这一路。

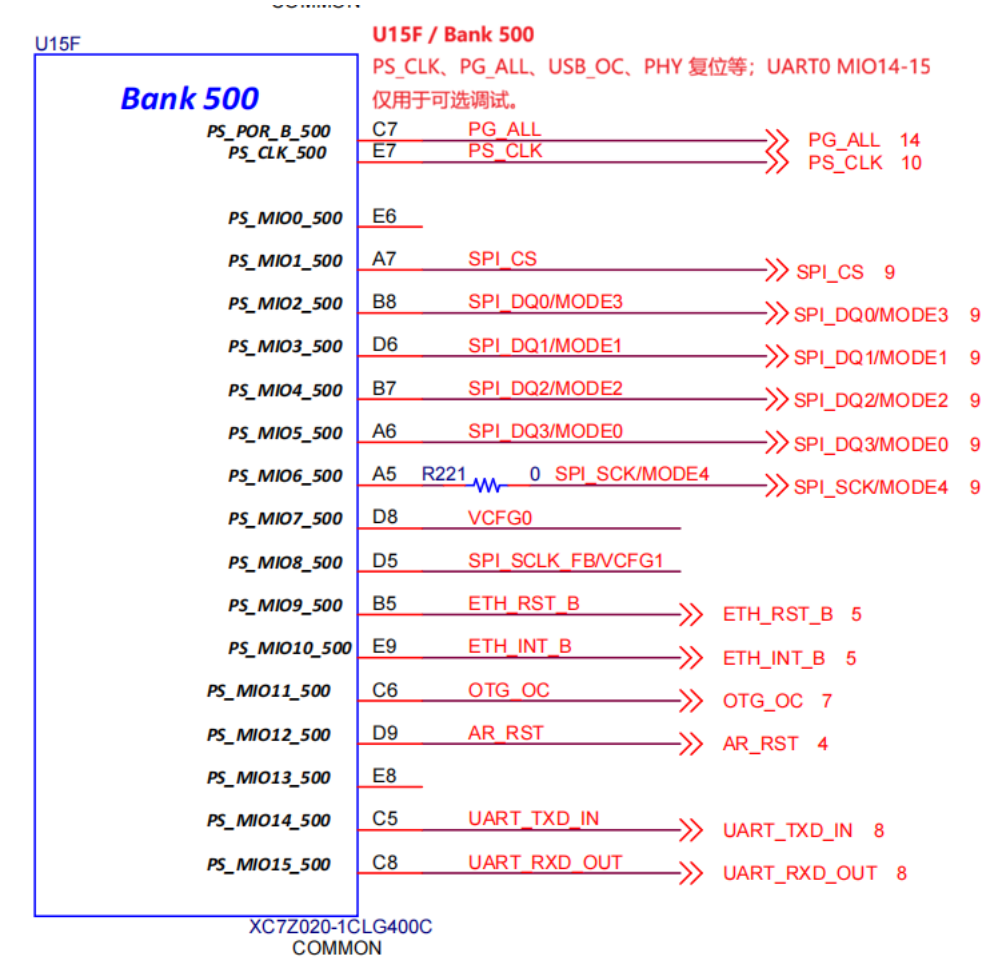
SD CARD

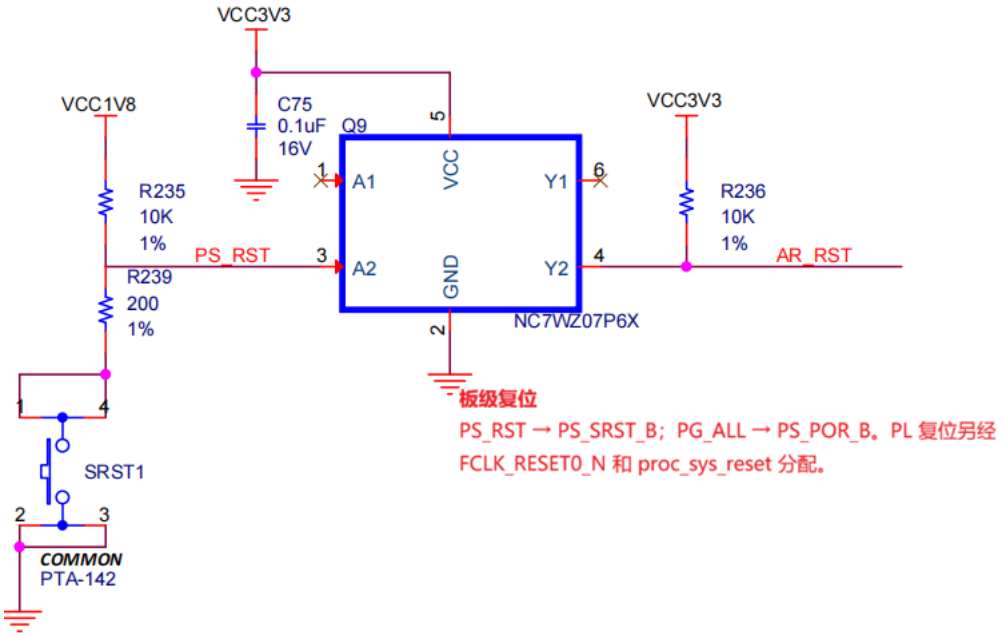


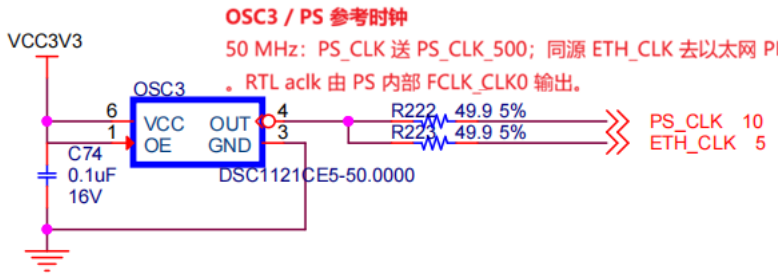

USB
USB_A_GND
USB_OTG_VBUS
USB_N
USB_P
USB_1
USB_4H
USB 1 / 摄像头接口
USB_P / USB_N 经 L2 等器件到 U9 的 USB_OTG_P / USB_OTG_N，摄像头插在此 USB Host 口。
OSC1 / USB 参考时钟
26 MHz，经 USB_OTG_RCLK 送入 U9，它不是自定义 RTL 的 50 MHz aclk。
U9 / USB PHY
TUSB1210BRHBR，ULPI：OTG_DATA[7:0]、CLK、DIR、NXT、STP 接 PS USB0 MIO28-39；RESETN 接 MIO46。
TUSB1210BRHBR
U10 / USB 供电开关
TPS2065：VCC5V0 → USB_OTG_VBUS → USB_1，OC 信号接 PS MIO11。
TPS2065DB

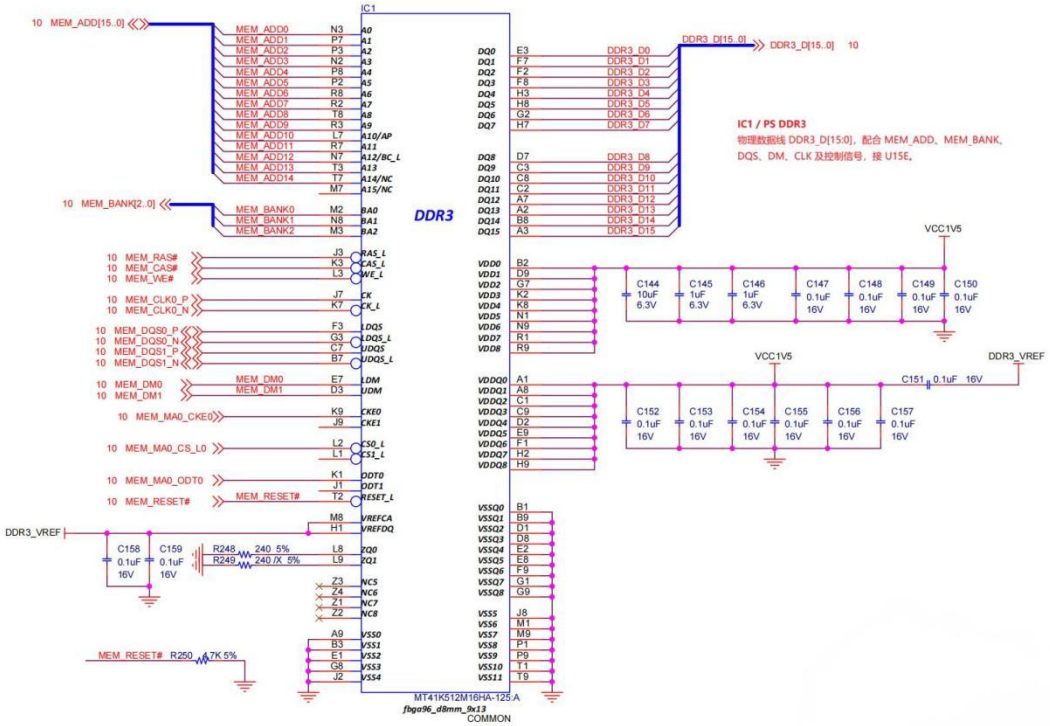

U15E
DDR PORT 502
XC7Z020-1CLG400C
U15E / DDR 专用端口
连接第 13 页 IC1。图像缓冲经过 PS DDR 控制器，不是直接把 DDR3 引脚接到 sobel_axis。
DDR3_VREF
MEM_ADD[15..0]
MEM_BANK[2..0]
MEM_RESET#
MEM_RAS#
MEM_CAS#
MEM_MA0_CS_L0
MEM_WE#
MEM_MA0_CKE0
MEM_CLK0_P
MEM_CLK0_N
MEM_MA0_ODT0
R218 80.6 1%
R219 80.6 1%
R220 80.6 1%
DDR_VRP
DDR_VRN
VCC1V5
DDR3_D[15..0]
MEM_DM1
MEM_DM0
MEM_DQS1_P
MEM_DQS1_N
MEM_DQS0_P
MEM_DQS0_N

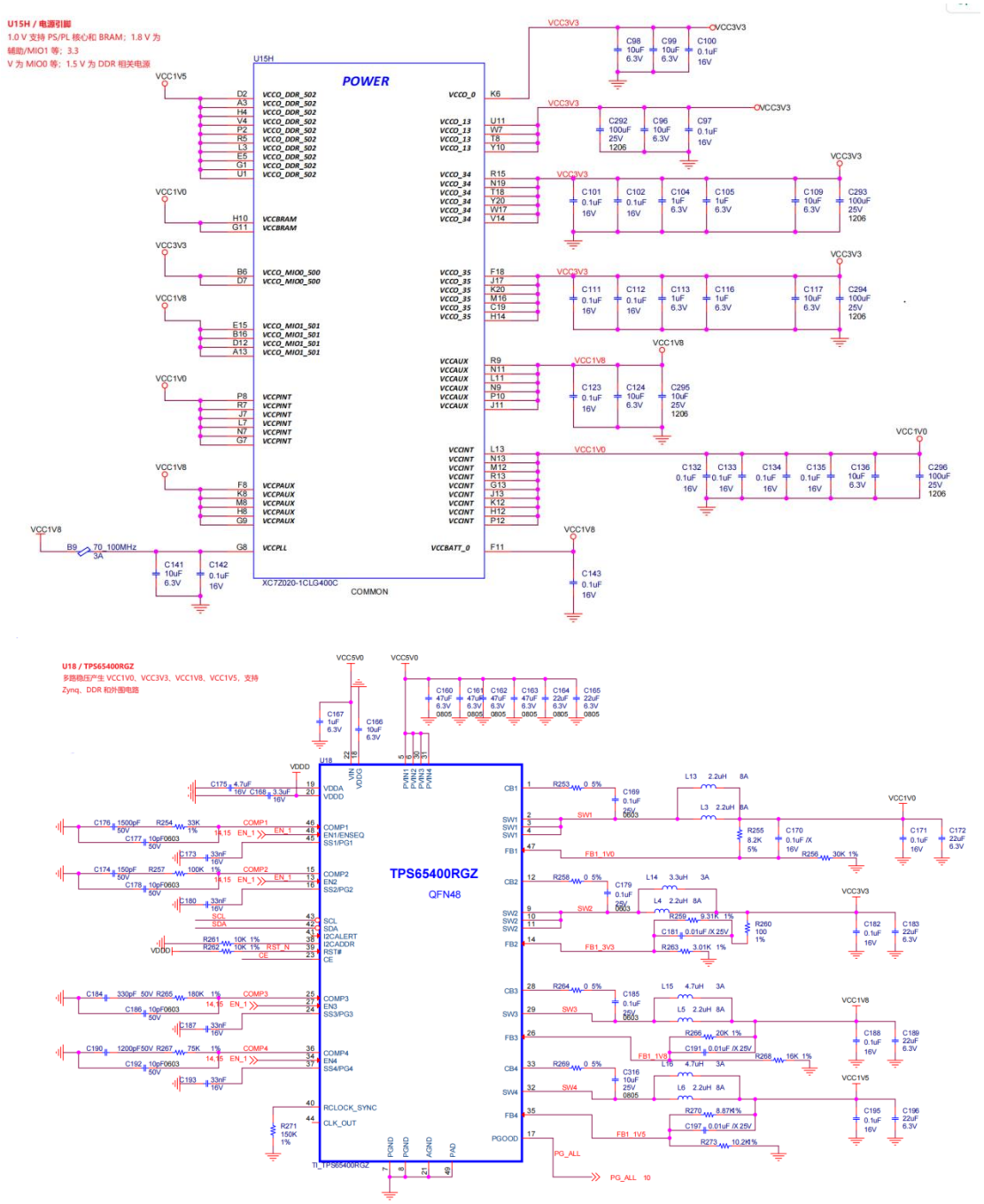

U15G
Bank 501
PS_SRST_B_501
PS_MIO_VREF_501
B10
E11
PS_RST
VREF0V9
U15G / Bank 501
PS MIO 外设:
Ethernet、USB、SD
，另含 ETH_MDC /
ETH_MDIO。对应
generated wrapper
的 FIXED_IO_mio。
PS_MIO16_501 A19 ETH_TXCK ETH_TXCK 5
PS_MIO17_501 E14 ETH_TXD0 ETH_TXD0 5
PS_MIO18_501 B18 ETH_TXD1 ETH_TXD1 5
PS_MIO19_501 D10 ETH_TXD2 ETH_TXD2 5
PS_MIO20_501 A17 ETH_TXD3 ETH_TXD3 5
PS_MIO21_501 F14 ETH_TXCTL ETH_TXCTL 5
PS_MIO22_501 B17 ETH_RXCK ETH_RXCK 5
PS_MIO23_501 D11 ETH_RXD0 ETH_RXD0 5
PS_MIO24_501 A16 ETH_RXD1 ETH_RXD1 5
PS_MIO25_501 F15 ETH_RXD2 ETH_RXD2 5
PS_MIO26_501 A15 ETH_RXD3 ETH_RXD3 5
PS_MIO27_501 D13 ETH_RXCTL ETH_RXCTL 5
PS_MIO28_501 C16 OTG_DATA4 OTG_DATA4 7
PS_MIO29_501 C13 OTG_DIR OTG_DIR 7
PS_MIO30_501 C15 OTG_STP OTG_STP 7
PS_MIO31_501 E16 OTG_NXT OTG_NXT 7
PS_MIO32_501 A14 OTG_DATA0 OTG_DATA0 7
PS_MIO33_501 D15 OTG_DATA1 OTG_DATA1 7
PS_MIO34_501 A12 OTG_DATA2 OTG_DATA2 7
PS_MIO35_501 F12 OTG_DATA3 OTG_DATA3 7
PS_MIO36_501 A11 OTG_CLK OTG_CLK 7
PS_MIO37_501 A10 OTG_DATA5 OTG_DATA5 7
PS_MIO38_501 E13 OTG_DATA6 OTG_DATA6 7
PS_MIO39_501 C18 OTG_DATA7 OTG_DATA7 7
PS_MIO40_501 D14 R217 40.2 5% SD_CCLK 5
PS_MIO41_501 C17 SD_CMD SD_CMD 5
PS_MIO42_501 E12 SD_D0 SD_D0 5
PS_MIO43_501 A9 SD_D1 SD_D1 5
PS_MIO44_501 F13 SD_D2 SD_D2 5
PS_MIO45_501 B15 SD_D3 SD_D3 5
PS_MIO46_501 D16 OTG_RESETN OTG_RESETN 7
PS_MIO47_501 B14 SD_CD SD_CD 5
PS_MIO48_501 B12
PS_MIO49_501 C12
PS_MIO50_501 B13 HDMI_TX_SCL HDMI_TX_SCL 6
PS_MIO51_501 B9 HDMI_TX_SDA HDMI_TX_SDA 6
PS_MIO52_501 C10 ETH_MDC ETH_MDC 5
PS_MIO53_501 C11 ETH_MDIO ETH_MDIO 5
XC7Z020-1CLG400C
COMMON

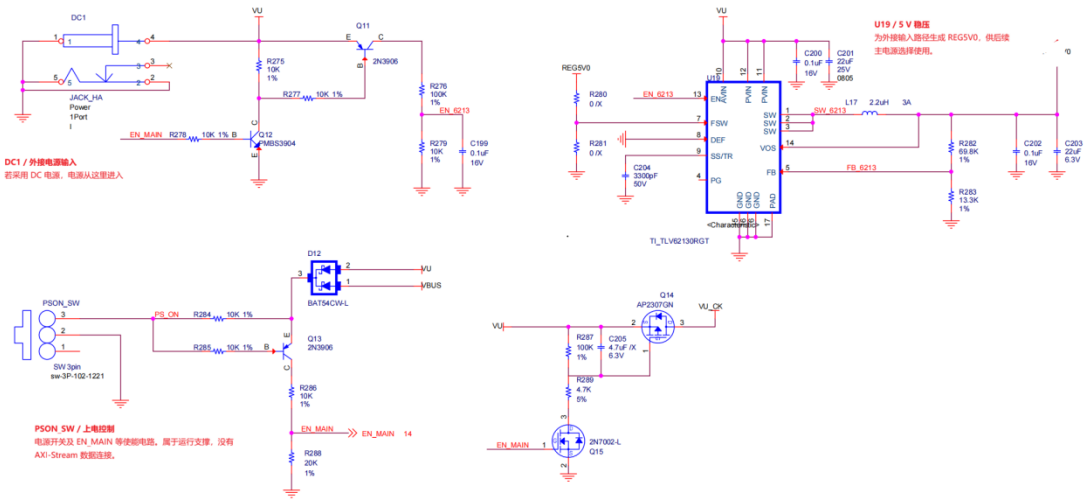

U15F
U15F / Bank 500
PS_CLK、PG_ALL、USB_OC、PHY 复位等；UART0 MIO14-15
仅用于可选调试。
Bank 500
PS_POR_B_500 C7 PG_ALL PG_ALL 14
PS_CLK_500 E7 PS_CLK PS_CLK 10
PS_MIO0_500 E6
PS_MIO1_500 A7 SPI_CS SPI_CS 9
PS_MIO2_500 B8 SPI_DQ0/MODE3 SPI_DQ0/MODE3 9
PS_MIO3_500 D6 SPI_DQ1/MODE1 SPI_DQ1/MODE1 9
PS_MIO4_500 B7 SPI_DQ2/MODE2 SPI_DQ2/MODE2 9
PS_MIO5_500 A6 SPI_DQ3/MODE0 SPI_DQ3/MODE0 9
PS_MIO6_500 A5 R221 0 SPI_SCK/MODE4 SPI_SCK/MODE4 9
PS_MIO7_500 D8 VCFG0
PS_MIO8_500 D5 SPI_SCLK_FB/VCFG1
PS_MIO9_500 B5 ETH_RST_B ETH_RST_B 5
PS_MIO10_500 E9 ETH_INT_B ETH_INT_B 5
PS_MIO11_500 C6 OTG_OC OTG_OC 7
PS_MIO12_500 D9 AR_RST AR_RST 4
PS_MIO13_500 E8
PS_MIO14_500 C5 UART_TXD_IN UART_TXD_IN 8
PS_MIO15_500 C8 UART_RXD_OUT UART_RXD_OUT 8
XC7Z020-1CLG400C
COMMON

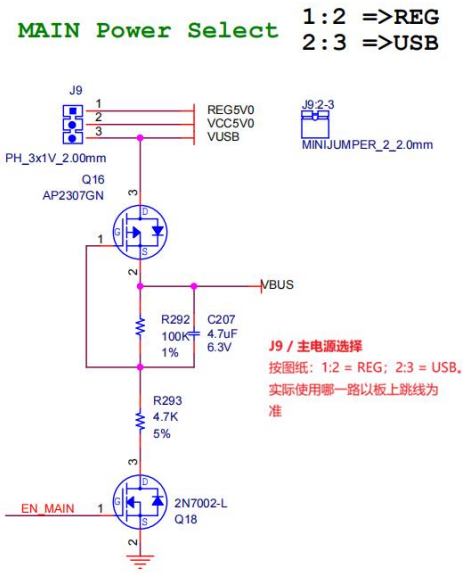

VCC3V3
VCC1V8
C75
0.1uF
16V
Q9
R235
10K
1%
R239
200
1%
PS_RST
A1
A2
VCC
GND
Y1
Y2
VCC3V3
R236
10K
1%
AR_RST
NC7WZ07P6X
SRST1
COMMON
PTA-142
板级复位
PS_RST → PS_SRST_B；PG_ALL → PS_POR_B。PL 复位另经
FCLK_RESET0_N 和 proc_sys_reset 分配。


OSC3 / PS 参考时钟
50 MHz：PS_CLK 送 PS_CLK_500；同源 ETH_CLK 去以太网 PHY
。RTL aclk 由 PS 内部 FCLK_CLK0 输出。
VCC3V3
OSC3
VCC
OUT
OE
GND
C74
0.1uF
16V
DSC1121CE5-50.0000
R222 49.9 5%
R223 49.9 5%
PS_CLK 10
ETH_CLK 5


IC1
DDR3
IC1 / PS DDR3
物理数据线 DDR3_D[15:0]，配合 MEM_ADD、MEM_BANK、DQS、DM、CLK 及控制信号，接 U15E。
MT41K512M16HA-125:A
fbga96_d8mm_9x13
COMMON
VCC1V5
DDR3_VREF


U15H / 电源引脚
1.0 V 支持 PS/PL 核心和 BRAM；1.8 V 为
辅助/MIO1 等；3.3
V 为 MIO0 等；1.5 V 为 DDR 相关电源
U15H
POWER
XC7Z020-1CLG400C
COMMON
U18 / TPS65400RGZ
多路稳压产生 VCC1V0、VCC3V3、VCC1V8、VCC1V5，支持
Zynq、DDR 和外围电路
TPS65400RGZ
QFN48





2.2.3.1 图像处理顶层与双路分发电路

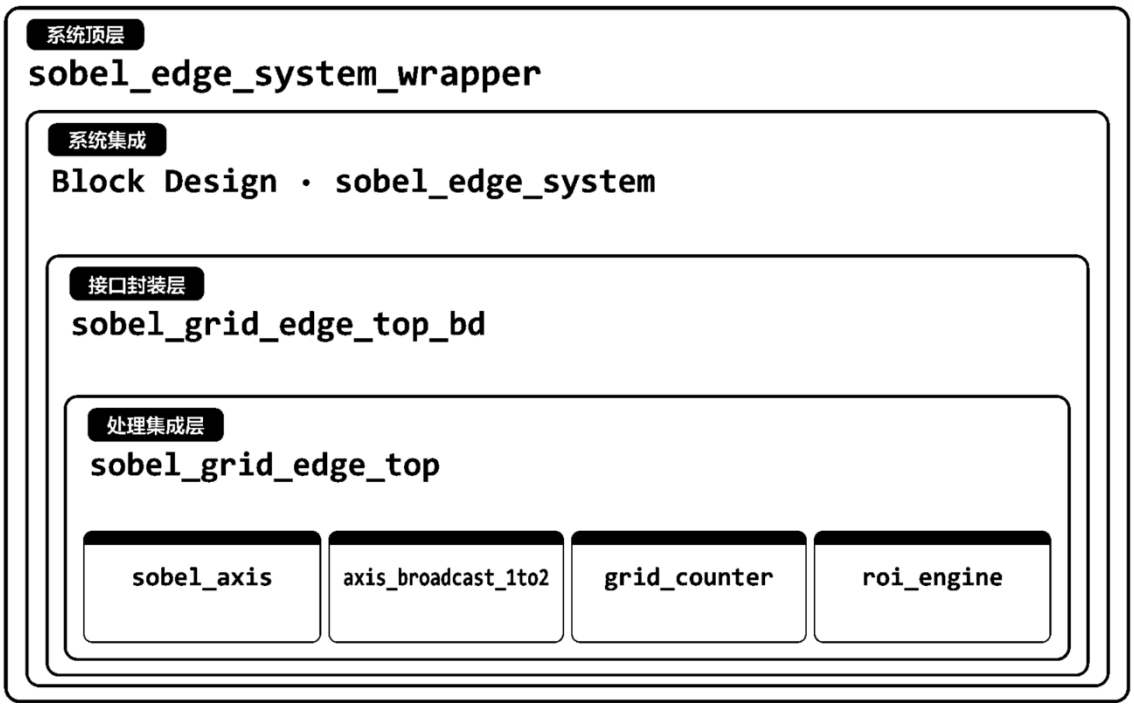
2.2.3.1.1 两级顶层结构

（1）模块功能介绍

图像处理顶层将各运算子模块连接为完整的处理通路，并向系统提供统一的输入、输出接口。设计采用两级自定义顶层：外层 sobel_grid_edge_top_bd 面向 Vivado Block Design，内层 sobel_grid_edge_top 面向算法电路集成。两层分别处理工具接口封装和内部模块连接，内部运算逻辑保持一致。

从整个工程来看，最上层为 Vivado 生成的 sobel_edge_system_wrapper，其内部 Block Design 包含 Zynq PS、两组 DMA、两组 AXI 互连、复位控制及

自定义处理模块(下图未标注)。两级自定义顶层位于该系统之内，因此“整个工程顶层”和“图像处理顶层”需要区分。下图给出了相应层次：



系统顶层与自定义 RTL 层级结构

（2）顶层与子模块关系

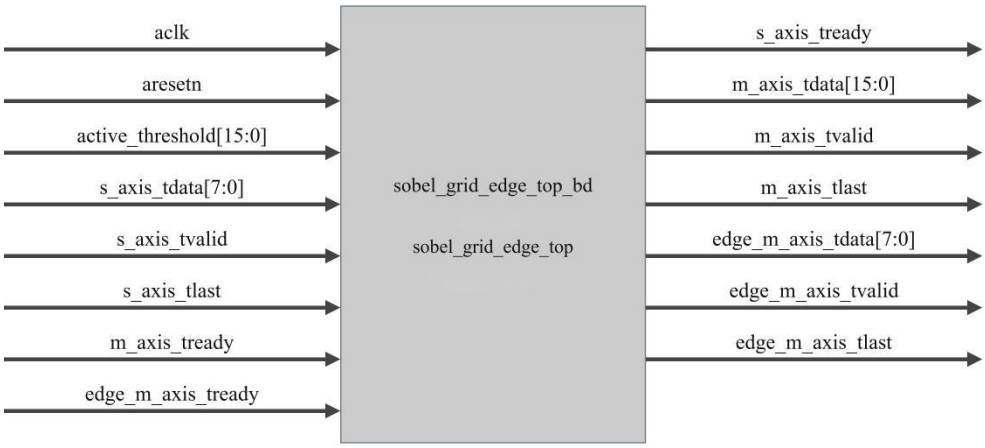
外层封装通过接口属性声明把数据、有效、就绪及结束信号组合为 AXI-Stream 总线，同时声明时钟、复位极性和接口关联。当前工程使用这一封装作为 Block Design 的 Module Reference。实际工作时钟由 PS 侧 FCLK_CLK0 提供。该层没有另设图像处理状态机，主要完成端口透传和接口识别。内层顶层例化 sobel_axis、axis_broadcast_1to2、grid_counter 和 roi_engine。灰度像素首先进入 Sobel 模块，输出的二值边缘流送入 AXIS Broadcast 模块；AXIS Broadcast 的 m0 通路连接网格统计，m1 通路连接边缘图输出。网格统计产生的 48 个计数继续送入 ROI 模块，最终得到六字段结果。

（3）模块接口设计

外层与内层顶层采用相同的数据端口，以同名连接传递信号。输入接口 s_axis 由 s_axis_tdata[7:0] 、 s_axis_tvalid 、 s_axis_tready 和 s_axis_tlast 组成，连接主 DMA 的 MM2S 输出。TDATA 表示当前灰度像素，TVALID 由 DMA 给出，TREADY 由处理链路返回。输入需要按固定行优先顺序提供完整帧，硬件不会接收单独的像素地址。

ROI 输出使用 m_axis_tdata[15:0]传递字段，配合 m_axis_tvalid、m_axis_tready 和 m_axis_tlast 与主 DMA 的 S2MM 接口连接。边缘图输出为 edge_m_axis_tdata[7:0]及对应的 valid、ready、last，连接第二路 DMA 的 S2MM。两个输出口有独立的就绪信号，因此可以分别接受接收端的暂停控制。

active_threshold[15:0] 是普通并行输入。所有自定义子模块使用同一工作时钟，复位为低有效。模块单元如下图，输入箭头指向模块，输出箭头离开模块；其中 m_axis_tready 和 edge_m_axis_tready 虽属于输出流的握手信号，方向仍是输入。



两级自定义顶层共用模块端口

接口名称	I/O	功能描述
aclk	I	模块时钟 50 MHz
aresetn	I	低电平同步复位
active_threshold[15:0]	I	ROI 活跃网格判定阈值

接口名称	I/O	功能描述
s_axis_tdata[7:0]	I	输入 DMA 提供的灰度像素
s_axis_tvalid	I	灰度像素有效
s_axis_tlast	I	输入帧末标志
m_axis_tready	I	ROI 接收 DMA 的就绪信号
edge_m_axis_tready	I	边缘图接收 DMA 的就绪信号
s_axis_tready	0	灰度像素接收就绪
m_axis_tdata[15:0]	0	每帧依次输出 x、y、w、h、最大四连通域活跃格数、面积缩减率
m_axis_tvalid	0	ROI 字段有效
m_axis_tlast	0	ROI 数据包结束标志
edge_m_axis_tdata[7:0]	0	二值边缘图
edge_m_axis_tvalid	0	边缘像素有效收
edge_m_axis_tlast	0	边缘图帧末标志

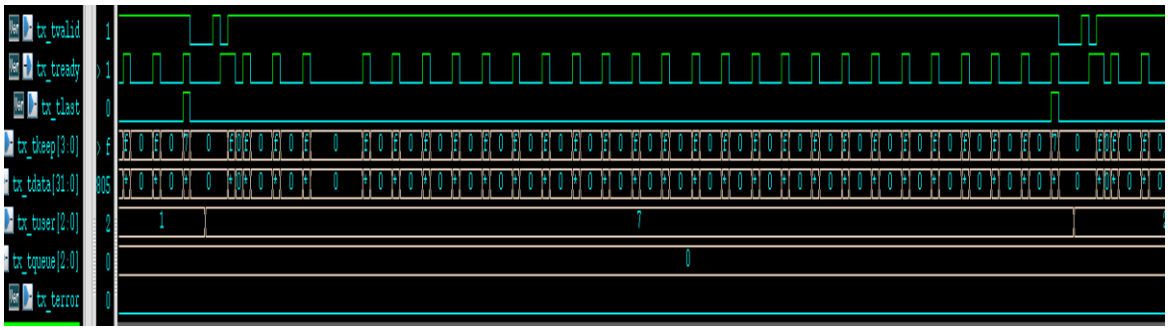
sobel_grid_edge_top 模块接口说明

2.2.3.1.2 流式接口标准时序及约定

（1）AXI-Stream 握手过程

发送端在数据准备好后给出 TDATA 并置位 TVALID,接收端通过 TREADY 表示能否接收。两者在同一上升沿均有效，才完成一次传输；发送端不能把接收端提前给出 TREADY 作为产生 TVALID 的必要条件。若 TVALID 已为高而 TREADY 为低，发送端保持 TVALID、TDATA 和 TLAST，直至该数据被接收。

下图中，P0 在上升沿 1、2 均未被接收，保持到上升沿 3 完成交接；P1 在上升沿 4 被接收；末数据 P2 在上升沿 5 遇到暂停，其数据和 TLAST 一同保持至上升沿 6。因此，TLAST 拉高并不意味着传输已经结束，必须同时完成握手。



AXI-Stream 握手协议仿真波形(TVALID 先于 TVALID 置高)

（2）数据包与结束标志

当前输入灰度流和输出边缘流均以整帧作为一个数据包，网格计数流以 48 个计数作为一个数据包，ROI 流以六字段作为一个数据包。它们的 TLAST 分别出现在整帧末像素、第 48 个计数和第六个字段。该约定不同于视频流接口中逐行给出 TLAST 的方式。

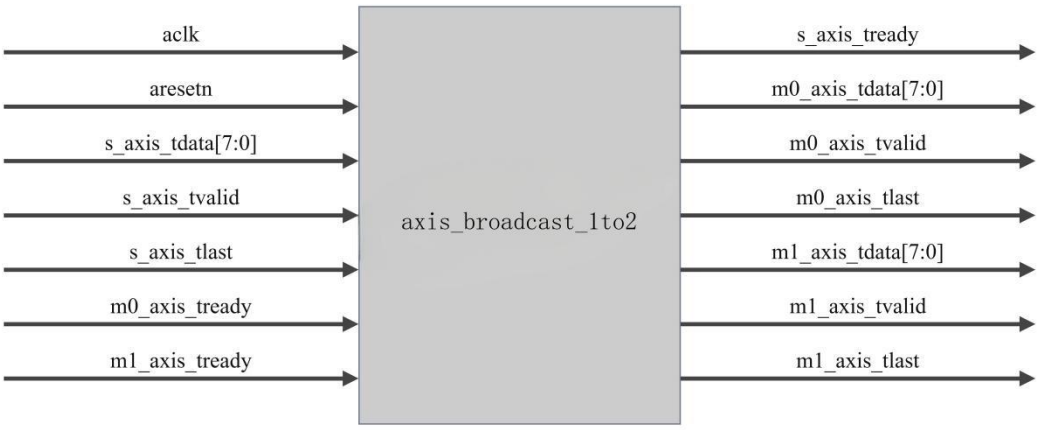
顶层不使用 TUSER、TID 或 TDEST。灰度和边缘接口的有效数据宽度为一字节，ROI 接口为两字节，均通过握手次数和固定顺序确定数据含义。Sobel 虽保留输入 TLAST 端口，当前坐标推进和输出结束判定实际依据固定宽高；ROI 接收端也按 48 次有效握手结束接收。

2.2.3.1.3 边缘数据 AXI-Stream broadcast 子模块

（1）模块功能与接口设计

边缘图回传与网格统计都需要使用 Sobel 的完整输出。若直接把同一组 valid 和 data 接到两条接收通路上，两端就绪状态不一致时，可能出现一端已经取走数据、另一端尚未接收的问题。为此，顶层设置一入两出的 Broadcast 子模块，为两条通路分别记录当前像素是否完成交接。

AXIS Broadcast 模块输入为一组 8 位流式接口，输出为 m0、m1 两组相同格式的流式接口。内部的 data_reg 和 last_reg 保存一份像素及结束标志，m0_axis_tvalid 和 m1_axis_tvalid 分别表示该数据是否仍等待相应接收端。



axis_broadcast_1to2 模块端口

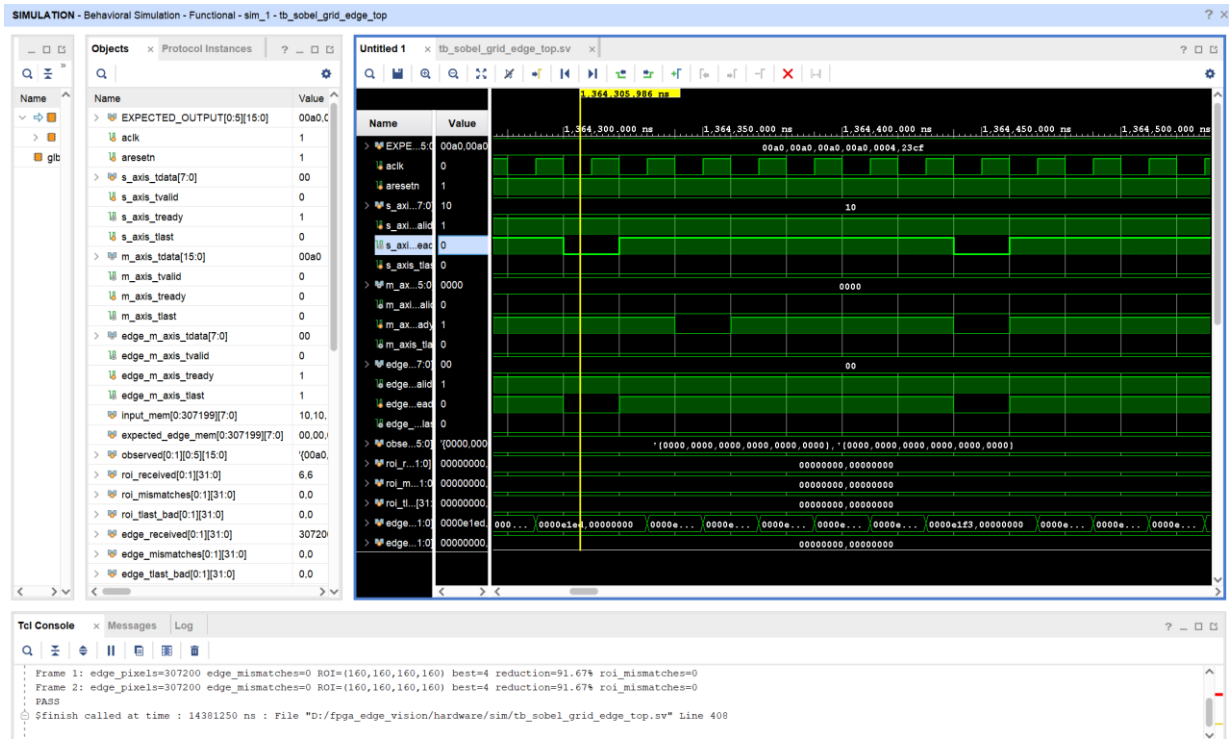
接口名称	I/O	功能描述
aclk	I	模块时钟 50 MHz
aresetn	I	低电平同步复位
s_axis_tdata[7:0]	I	输入 DMA 提供的灰度像素
s_axis_tvalid	I	输出像素有效
s_axis_tlast	I	边缘图帧末标志
m0_axis_tready	I	网格统计模块接收就绪
m1_axis_tready	I	边缘图 DMA 接收就绪
s_axis_tready	0	两路旧像素均已接收
m0_axis_tdata[7:0]	0	发给 grid_counter 的二值边缘像素
m0_axis_tvalid	0	网格统计路像素有效
m0_axis_tlast	0	送往网格统计的帧末标志
m1_axis_tdata[7:0]	0	发送给边缘图 DMA 的二值边缘像素
m1_axis_tvalid	0	边缘图分支像素有效
m1_axis_tlast	0	送往边缘图 DMA 的帧末标志

axis_broadcast_1to2 模块接口说明

（2）分发控制逻辑

新数据被 AXIS broadcast 模块接收后，两路有效信号同时置位。若 m0 先完成握手而 m1 暂停，m0 的有效信号清零，m1 保持有效，缓存数据保持不变。此时 m0 不会重复接收旧像素，m1 仍可在后续时钟接收同一数据。只有两路旧数据都已完成或都能在当前拍完成交接，输入端才允许接收下一份数据。

系统在 Sobel 输出端设置 AXIS Broadcast 模块，将二值边缘流分为两路：一路进入网格统计与 ROI 筛选，结果通过主 DMA 接收通道回传；另一路通过边缘 DMA 回传完整边缘图。Broadcast 模块分别管理两路输出的握手状态，保留尚未被接收的数据，协调下游停顿时的数据传输，本模块仿真结果如下：



在 tcl 窗口中我们可以得到：

```
Frame 1: edge_pixels=307200 edge_mismatches=0
ROI=(160,160,160,160) best=4 reduction=91.67%
roi_mismatches=0
Frame 2: edge_pixels=307200 edge_mismatches=0
ROI=(160,160,160,160) best=4 reduction=91.67%
roi_mismatches=0
PASS
```

仿真顺利通过。

2.2.3.2 Sobel 边缘检测电路

（1）模块功能介绍

Sobel 电路接收 8 位灰度像素，计算局部灰度在水平和垂直方向的变化，再输出阈值化后的二值边缘。该结果是网格统计的输入，也直接用于边缘图回传。设计同时保留输入、输出图像的尺寸，最外圈像素输出为零，使后级能够继续按 640×480 的固定位置关系处理。

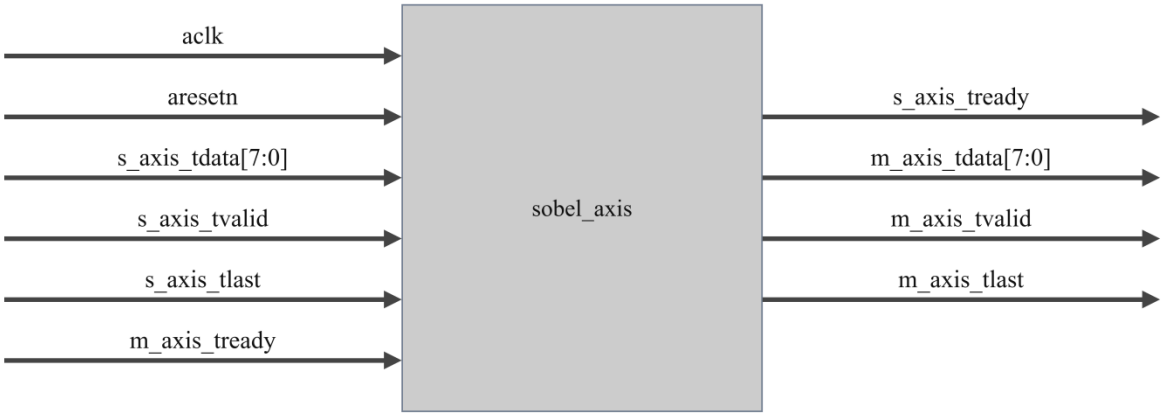
输入流每次只提供一个像素，而 Sobel 运算需要三行三列邻域。因此，电路先利用两块行缓存取得历史行数据，再用列移位寄存器组成窗口。

（2）模块接口设计

sobel_axis 的模块单元如下图所示。输入 s_axis_tdata[7:0] 连接主 DMA 经顶层送入的灰度流。模块仅在 s_axis_tvalid 与 s_axis_tready 同时有效时接收真实像素。s_axis_tready 在内部处理补列、补行时拉低，也会随输出阻塞而暂停，所以上游必须遵守握手。

输出 m_axis_tdata[7:0]为 0 或 255，m_axis_tvalid 表示结果有效，m_axis_tready 来自 Broadcast 子模块，m_axis_tlast 与本帧最后一个输出像素对应。该接口没有单独输出梯度值或像素坐标，坐标只在内部用于边界判断，并与像素同步经过流水线。

aclk 为当前 50 MHz 工作时钟，aresetn 在上升沿被检测为低时清除状态、坐标及各级有效标志。



Sobel 边缘检测模块单元

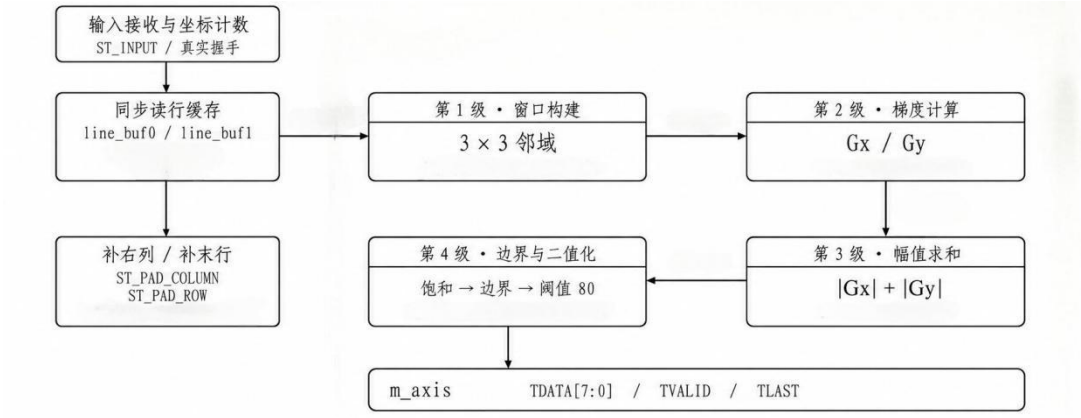
接口名称	I/O	功能描述
aclk	I	模块时钟 50 MHz
aresetn	I	低电平同步复位
s_axis_tdata[7:0]	I	灰度输入像素
s_axis_tvalid	I	输入灰度像素有效
s_axis_tlast	I	输入帧末标志接口
m_axis_tready	I	Broadcast 接收就绪
s_axis_tready	0	复位释放
m_axis_tdata[7:0]	0	8 位二值边缘像素
m_axis_tvalid	0	二值化流水线输出有效
m_axis_tlast	0	内部生成的输出帧末标志

sobel_axis 模块接口说明

（3）内部电路与数据对齐

两块行缓存 line_buf0、line_buf1 均为 640 深度、8 位宽。输入当前行像素时，同一列位置的历史数据同步读出；较近历史行的数据移入较早行缓存，当前像素写入较近行缓存，从而持续保存前两行。缓存采用同步读方式，读地址给出后需要一拍取得数据，所以当前像素、坐标、读写使能等都要延迟相应周期，然后才参与窗口的构建。

三行数据各保留前两列，加上新到的一列，就可以形成当前的 3×3 邻域。由于计算某个中心像素需要先取得其右侧和下一行邻点，输出中心位置对应内部读坐标-1。



Sobel 行缓存与四级运算流水结构

（4）运算原理与流水设计

①sobel 算子原理

Sobel 算子通过邻域像素的加权差分估计图像梯度，在检测灰度变化的同时，利用垂直于差分方向的加权平滑减轻噪声影响。本设计采用 3×3 Sobel 算子，规定图像坐标的 x 轴向右、y 轴向下，其水平、垂直方向的核分别为：

$$K_x=\begin{bmatrix}-1 & 0 & 1\\ -2 & 0 & 2\\ -1 & 0 & 1\end{bmatrix},\quad K_y=\begin{bmatrix}-1 & -2 & -1\\ 0 & 0 & 0\\ 1 & 2 & 1\end{bmatrix}$$

设窗口内像素为 p_{ij}（$i,j=0,1,2$，分别表示行、列），中心像素为 p_{11}，则两个方向的梯度分量为：

$$G_x = (p_{02} + 2p_{12} + p_{22}) - (p_{00} + 2p_{10} + p_{20})$$
$$G_y = (p_{20} + 2p_{21} + p_{22}) - (p_{00} + 2p_{01} + p_{02})$$

其中，G_x 反映左右灰度变化，对竖直边缘响应较强；G_y 反映上下灰度变化，对水平边缘响应较强。为简化硬件运算，本设计采用两个分量绝对值之和近似欧氏梯度幅值，并将结果饱和至 8 位范围：

$$M = |G_x| + |G_y|, \quad M_s = \min(M, 255)$$

随后以固定阈值 80 进行二值化，并将图像最外圈像素置零，得到边缘图：

$$E(x,y) = \begin{cases} 255, & \text{内部像素且 } M_s(x,y) \geq 80 \\ 0, & \text{其他情况} \end{cases}$$

②四级流水的任务划分

窗口数据准备好后，运算划分为四级。

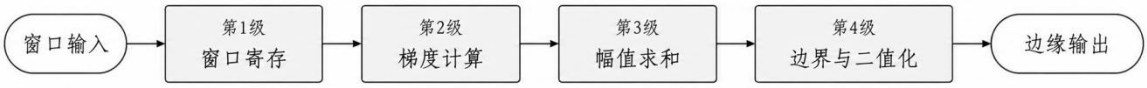
第一级寄存参与计算的八个邻域像素，以及中心坐标、有效标志和帧末标志。该级将行缓存与窗口形成过程同后续算术运算衔接起来；

第二级根据第一级窗口计算左右列、上下行加权和，完成 Gx、Gy，并将结果写入有符号寄存器。坐标及控制标志同时延迟一拍，保持与梯度对应；

第三级对第二级的两个梯度取绝对值并求和，将 M1 写入 14 位幅值寄存器。绝对值计算独立于前一级差分，有助于缩短单级组合通路；

第四级使用第三级幅值进行饱和、阈值比较和边界判断。边界判断使用随数据传来的中心坐标。帧末标志也在这一阶段到达输出，确保它与末像素保持一致。

四级流水处理流程

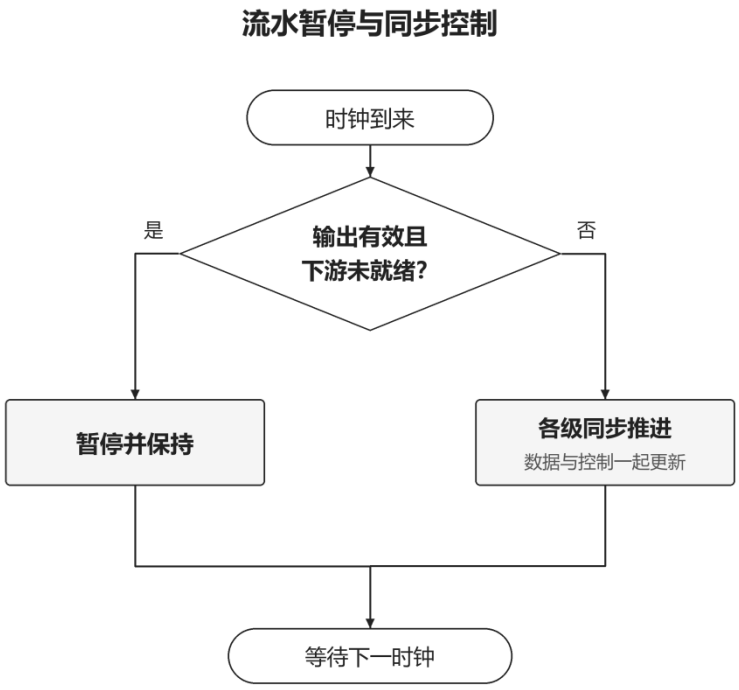


③流水暂停与数据、控制同步

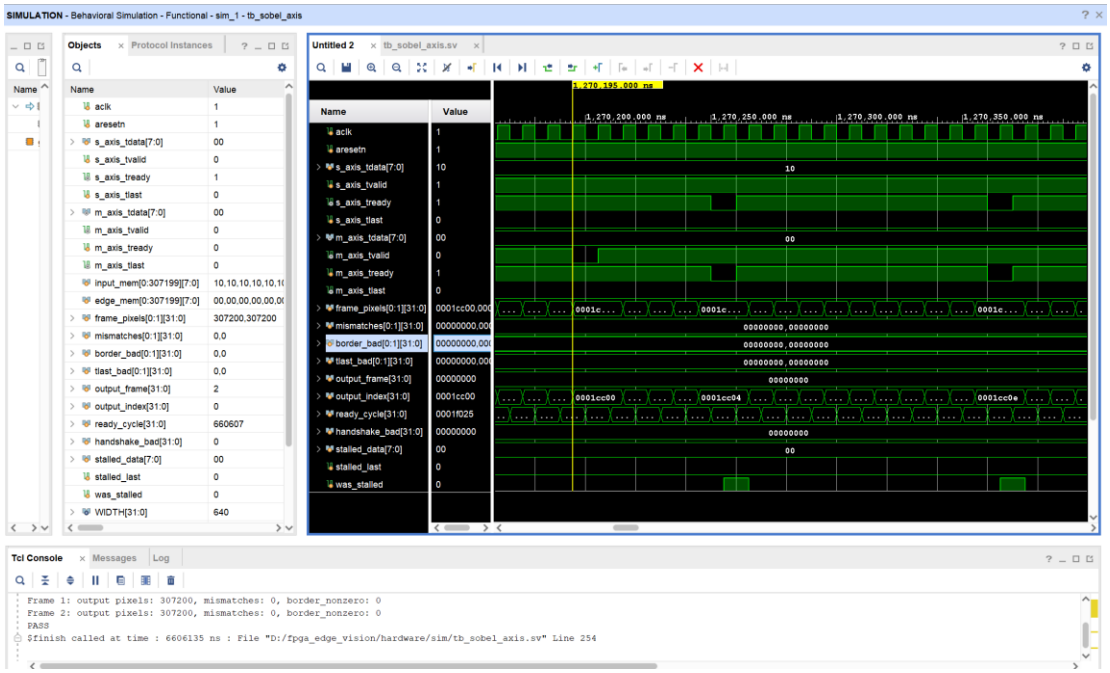
本模块的输出连接 broadcast 电路。当仍有未交付的数据时，可能拉低 m_axis_tready，使 Sobel 暂停。若输出级已经有效，此时继续更新窗口或梯度寄存器，就会覆盖尚未被下游取走的数据。

流水线推进使能由输出级有效状态与下游就绪状态共同决定。输出级为空时，可以向后推进已有计算；输出级有效且接收端就绪时，当前结果被取走，也允许装入下一结果。输出级有效而接收端未就绪时，各级数据、坐标、有效位和结束标志共同保持，行缓存的相关读写也暂停。在真实输入状态下，输入就绪信号同时受这一条件限制，使暂停能够传回 DMA。

若流水允许推进但当前没有有效输入，则通过有效位区分空拍与真实像素，避免把寄存器中残留的数据重复解释为新结果。行末补列与帧末补行仍由内部状态机产生，其作用在下一部分说明。由此，算术运算、接口握手和图像位置控制在同一推进规则下配合工作。



模块仿真验证：



在 tcl 窗口中我们可以得到：

```
Frame 1: output pixels: 307200, mismatches: 0,
border_nonzero:0
```

```
Frame 2: output pixels: 307200, mismatches: 0,
border_nonzero:0

PASS
```

说明仿真顺利通过。

2.2.3.3 网格边缘统计电路

（1）模块功能介绍

网格统计将二值边缘图划分为 8 列、6 行，每格为 80×80 像素，分别统计其中的边缘数量。统计结果表示局部结构信息的密集程度，后续 ROI 模块只需根据 48 个计数判断活跃区域，不必再遍历全部像素。

本模块由位置计数、像素比较、当前带累加、整帧结果保存和输出控制构成。它不保存完整边缘图，而是在输入过程中直接累计当前 80 行内的 8 个网格，完成一带后保存结果，再继续统计下一带。

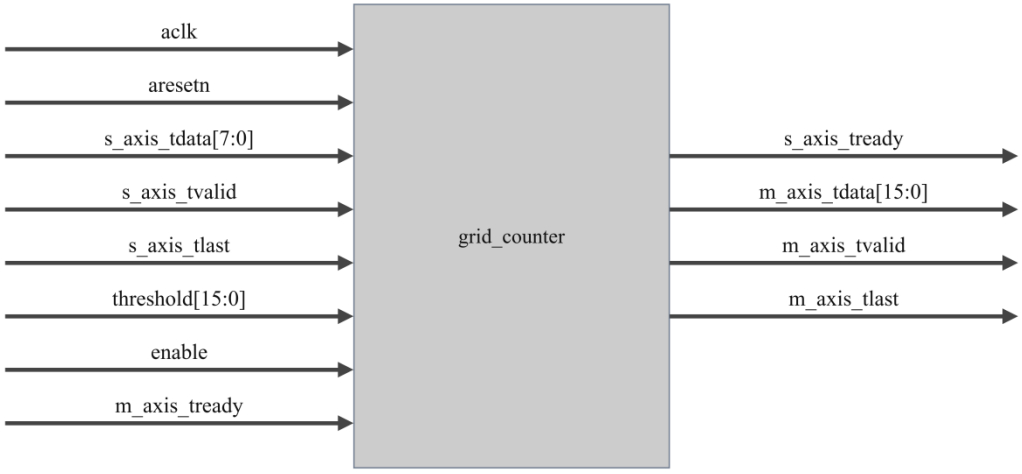
（2）模块接口设计

输入 s_axis_tdata[7:0] 连接 broadcast 模块的 m0 通路。s_axis_tvalid 和 s_axis_tready 完成握手时，电路处理当前像素并更新位置。输入 TLAST 用于结束当前帧并保存最后一带，因此它必须随像素一起交付，不能作为逐行结束标志使用。

threshold[15:0] 用于判定当前像素是否计入边缘数量，比较前将 8 位输入零扩展至 16 位；enable 只控制边缘命中，不停止位置计数或输出状态机。当前顶层把 threshold 接 80、enable 接 1，在这种情况下 255 被计数、0 不计入。但要注意的是，我们只是在这里把阈值固定为 80，而不是把该阈值作为判断整个网格是否活跃的阈值，阈值的动态变化是在软件端实现的。

输出 m_axis_tdata[15:0] 按行优先顺序发送 48 个网格计数，m_axis_tvalid 在输出状态有效，m_axis_tlast 在 output_idx 等于 47 时有

效。下游 roi_engine 未就绪时，输出索引不变，当前计数和结束标志保持。模块在输出阶段不再接收新像素，最后一个计数完成握手后清除统计数据，返回接收状态。



网格边缘统计模块单元

接口名称	I/O	功能描述
aclk	I	模块工作时钟 50 MHz
aresetn	I	低电平同步复位
s_axis_tdata[7:0]	I	来自 Broadcast 第 0 路的二值边缘像素
s_axis_tvalid	I	输入边缘像素有效
s_axis_tlast	I	整帧末像素标志态
threshold[15:0]	I	边缘像素计数阈值
enable	I	边缘计数使能
m_axis_tready	I	ROI 模块的计数接收就绪信号

接口名称	I/O	功能描述
s_axis_tready	0	复位释放且处于像素接收状态时为 1；输出计数期间为 0，暂停接收下一帧
m_axis_tdata[15:0]	0	边缘像素计数
m_axis_tvalid	0	网格计数输出有效
m_axis_tlast	0	在第 48 个网格计数有效时置 1；该项握手后清零计数并开始接收下一帧

grid_counter 模块接口说明

（3）统计电路结构与算法设计

用边缘标志 B 表示像素是否满足计数条件，则单格计数为：

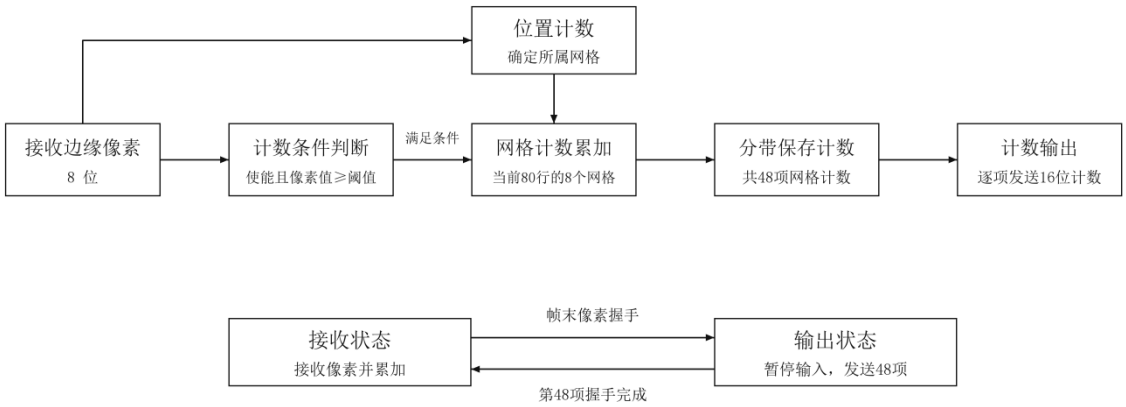
$$C_{r,c} = \sum_{j=0}^{79}\sum_{i=0}^{79} B\,(80c + i,\ 80r + j), \qquad r = 0,\ldots,5, \quad c = 0,\ldots,7.$$

其中，B 在 enable 有效且输入像素不小于 threshold 时取 1，其余为 0。

输出采用 16 位计数，便于与 ROI 输入接口直接连接：

```
assign edge_hit = enable && ({8'd0, s_axis_tdata} >=
threshold);
```

位置跟踪通过四个寄存器完成：x_in_grid 记录当前格内横向位置，grid_x 记录当前列网格，y_in_grid 记录当前带内行位置，grid_y 记录当前带编号，它们也只在实际接收像素时推进。

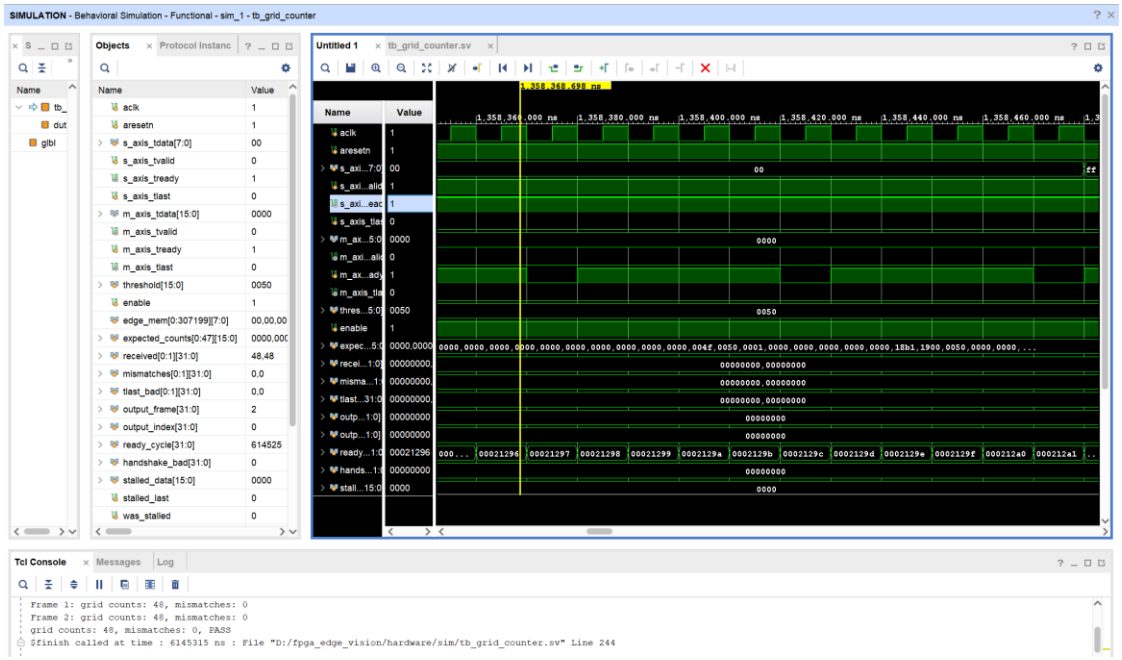


网格计数内部结构与收发状态

当前带使用 row_acc[0:7] 八个累加器。同一图像行依次经过八个网格，相应像素根据 grid_x 进入对应累加器；下一行仍累加到同一组寄存器，直到第 80 行结束。此时将结果写入 grid_counts 的对应八个位置，再清空 row_acc。

电路将整帧结果写入与读出分为两个状态，避免新帧累加与旧帧发送同时操作同一组数据。如果下游暂停接收计数，暂停经输入 ready 及 Broadcast 通路向上游传。

本模块仿真结果如下：



在 tcl 窗口中我们可以得到：

```
Frame1:ROI=(160,160,160,160),best_grids=4,reduction=91.67%,mismatches=0
Frame2:ROI=(160,160,160,160),best_grids=4,reduction=91.67%,mismatches=0
PASS
```

说明仿真顺利通过。

2.2.3.4 四连通动态 ROI 筛选电路

（1）模块功能介绍

ROI 电路把网格计数转换为活跃状态，搜索所有四连通区域，并输出活跃网格数量最多区域的外接矩形。其硬件处理对象是 48 位网格图，不是原始图像像素；区域搜索所需的寄存和按位运算规模因此受到限制。

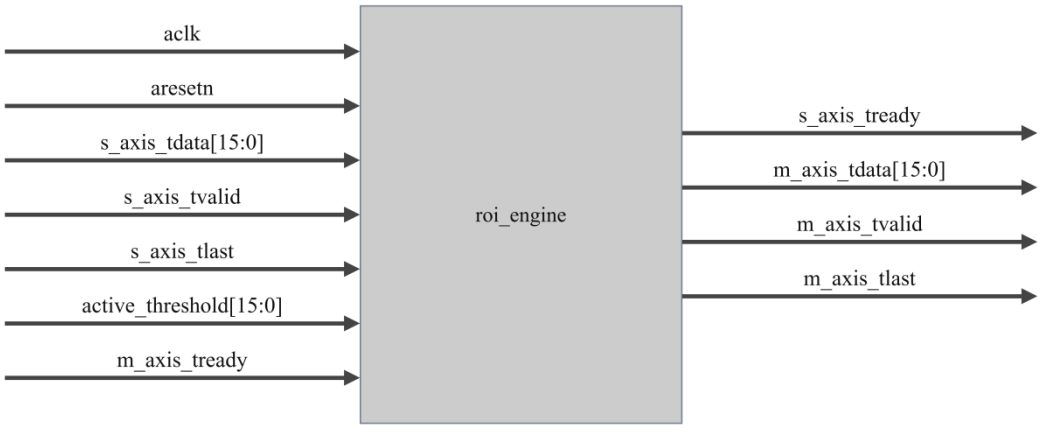
“动态”表示选中区域的位置和尺寸随每帧内容变化。当前活跃阈值由外部常量提供，并没有自动阈值调整电路。筛选依据是边缘数量和空间连接关系，不能直接将选中区域等同于车牌或其他目标。

（2）模块接口设计

模块输入 s_axis_tdata[15:0] 接收 48 个计数。每次输入握手成功，count_index 指向的 active_map 位更新为当前比较结果。计数按固定顺序到达，模块无需接收额外网格坐标。当前 RTL 以 count_index 达到 47 作为接收结束条件，保留的 s_axis_tlast 端口未用于重新界定计数长度。

active_threshold[15:0] 连接顶层输入。接收完成后，s_axis_tready 拉低，模块依次执行初始化、区域搜索和结果换算，期间不接收下一帧计数。输出 m_axis_tdata[15:0] 按 x、y、w、h、best_grids、reduction_x100 顺序发送，输出索引仅随握手递增，第六字段携带 TLAST。下游暂停时，当前字段保持。

输出中没有独立的 roi_valid 端口，也没有并行输出六字段的接口。无活跃网格时，坐标、宽高和 best_grids 均为零，reduction_x100 为 10000；六字段仍照常发送，使 DMA 每帧接收长度固定。结果坐标用于软件裁剪，本电路不读取或裁剪彩色原图。



动态 ROI 筛选模块单元

接口名称	I/O	功能描述
aclk	I	模块工作时钟 50 MHz
aresetn	I	低电平同步复位
s_axis_tdata[15:0]	I	来自 grid_counter 的网格边缘像素计数
s_axis_tvalid	I	网格计数有效
s_axis_tlast	I	输入计数包结束标志接口
active_threshold[15:0]	I	ROI 活跃网格判定阈值
m_axis_tready	I	主 DMA 的 ROI 接收就绪信号
s_axis_tready	O	复位释放且处于计数接收状态时为 1；连通域搜索、ROI 计算及结果输出期间为 0
m_axis_tdata[15:0]	O	每帧依次输出 x、y、w、h、最大四连通域活跃格数、面积缩减率

接口名称	I/O	功能描述
m_axis_tvalid	0	ROI 结果字段有效
m_axis_tlast	0	面积缩减率有效时置 1；该字段握手后结束

roi_engine 模块接口说明

（3）ROI 筛选模块算法设计

ROI 筛选模块负责将网格边缘统计结果转换为原图中的矩形候选区域。在本项目的处理链路中，Sobel 模块提取图像边缘，grid_counter 模块统计各网格内的边缘像素数量，roi_engine 模块根据网格的活跃程度与空间连接关系确定 ROI。筛选结果通过 DMA 回传至 PS 端，由软件完成原图裁剪，为后续显示、抓拍和识别提供候选图像。模块的核心处理包括活跃网格判定、四连通区域搜索、主要区域选择以及矩形坐标生成。

① 网格计数接收与活跃判定

模块按行顺序接收 grid_counter 输出的 48 个网格计数。在输入接口完成有效握手时，将当前计数与活跃阈值比较，并更新对应网格的活跃标志：

```
active_map[count_index] <= (s_axis_tdata > active_threshold);
```

其中，s_axis_tdata 为当前网格的边缘像素数量，active_threshold 为活跃阈值，当前工程设置为 768；active_map 保存整帧 8×6 个网格的判定结果。设网格计数为 $n_{r,c}$，则活跃判定可表示为：

$$A_{r,c} = \begin{cases} 1, & n_{r,c} > T \\ 0, & n_{r,c} \le T \end{cases}$$

该比较采用严格大于关系，计数等于阈值时判为非活跃。模块在接收过程中逐项生成活跃位图，接收完一帧的 48 个网格计数后进入区域搜索阶段。由此，后续搜索处理的是网格的空间分布，而每个网格的原始边缘计数已通过阈值判定体现。

② 基于四邻域扩张的连通区域搜索

活跃网格可能分布在目标轮廓、背景纹理和局部干扰中，需要结合空间连接关系将其组织为候选区域。模块采用四连通规则，将上、下、左、右相邻的活跃网格归入同一区域。

搜索从尚未访问的活跃网格开始。frontier_map 表示当前一轮待纳入区域的网格，component_map 表示已纳入当前区域的网格，neighbor_map 则由当前前沿向四个方向移位生成。下一轮扩张前沿的计算为：

```
assign new_frontier = neighbor_map & active_map
                    & ~(component_map | frontier_map);
```

依次完成两个筛选：通过 active_map 保留活跃邻格，再排除已经纳入区域或正在处理的网格，从而避免重复访问。邻域生成时，模块对左右移位结果施加列边界掩码，防止一行末端与下一行起点错误连接。

每轮扩张同时更新区域位图和活跃网格数量：

```
component_map  <= component_map | frontier_map;
component_size <= component_size + frontier_popcount;
```

其中 frontier_popcount 为本轮前沿中活跃网格的数量。由于位图操作可以同时处理多个网格，一轮扩张能够收集多个相邻位置。当 new_frontier 为空时，当前连通区域搜索结束，模块再从剩余未访问网格中选择新的起点。

③ 主要连通区域选择与边界保存

每完成一个连通区域的搜索，模块将其活跃网格数 component_size 与当前最大值 best_size 比较：

```
if (component_size > best_size) begin
    best_size  <= component_size;
    best_min_x <= comp_min_x;
    best_max_x <= comp_max_x;
    best_min_y <= comp_min_y;
    best_max_y <= comp_max_y;
end
```

当当前的区域规模更大时，模块同步保存其活跃网格数量及行列边界。这里的筛选依据是连通区域包含的活跃网格数。我们采用严格大于判断，数量相等时保留此前结果。配合从低索引开始的行优先搜索顺序，多个等规模区域将选择最先发现的区域，使相同输入得到确定的筛选结果。

④ 外接矩形生成与像素坐标映射

搜索过程中，模块持续记录当前区域覆盖的最小、最大行列坐标。选出主要区域后，利用这些边界生成覆盖该区域的轴对齐外接矩形。

设单个网格的宽、高分别为 g_w、g_h，所选区域的列边界为 c_{min}、c_{max}，行边界为 r_{min}、r_{max}，则：

$$\begin{aligned} x_{ROI} &= c_{min} g_w \\ y_{ROI} &= r_{min} g_h \\ w_{ROI} &= (c_{max} - c_{min} + 1) g_w \\ h_{ROI} &= (r_{max} - r_{min} + 1) g_h \end{aligned}$$

对应的坐标输出代码为：

```
roi_x <= scale_by_80({1'b0, best_min_x});

roi_y <= scale_by_80({1'b0, best_min_y});

roi_w <= scale_by_80(best_span_x);

roi_h <= scale_by_80(best_span_y);
```

其中，best_span_x、best_span_y 表示外接矩形横向和纵向覆盖的网格数；scale_by_80 对应本项目采用的固定网格尺寸。

```
function automatic [15:0] scale_by_80(input [3:0] value);
     reg [15:0] extended_value;
     begin
      extended_value = {12'd0, value};
 scale_by_80 = (extended_value << 6) + (extended_value << 4);
   end
Endfunction
```

由此得到的位置和尺寸可以直接交给 PS 端用于裁剪。对于形状不规则的连通区域，外接矩形内部可能包含非活跃网格，因此，矩形覆盖范围与连通区域的活跃网格数量分别保存。

⑤ 面积评估、结果回传与验证

模块通过面积缩减率描述 ROI 裁剪范围相对于整幅图像的面积变化。设图像宽、高为 W、H，则：

$$\eta = 1 - \frac{w_{ROI} h_{ROI}}{WH}$$

硬件先计算外接矩形覆盖的网格数量，再通过查找表获得定点形式的缩减率：

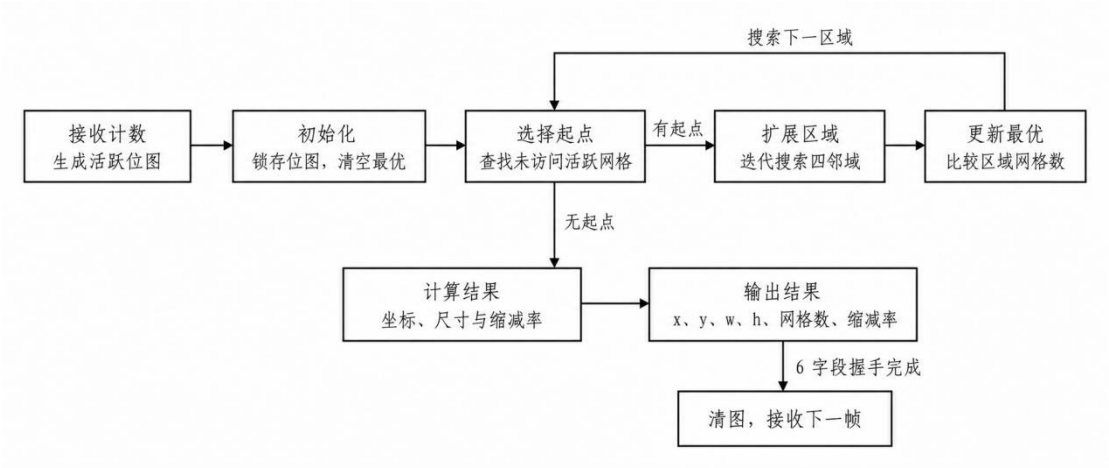
```
reduction_x100 <= REDUCTION_LUT[bbox_grid_count_reg];
```

其中，bbox_grid_count_reg 保存外接矩形覆盖的总网格数。它与用于区域选择的 best_size 含义不同：前者用于计算裁剪面积，后者用于衡量连通区域规模。结果计算完成后，模块依次输出 x、y、宽、高、活跃网格数和面积缩减率，共 6 个 16 位字段，并在最后一个字段置位 TLAST。

配套测试平台 tb_roi_engine.sv 连续发送两帧单目标数据，并追加一帧包含三个等规模区域的定向用例，用于检查矩形生成、连续帧处理及区域选择规则。定向用例还将一个网格计数设置为 768，检查计数等于阈值时不会被判为活跃。同时，通过以下语句持续施加输出反压：

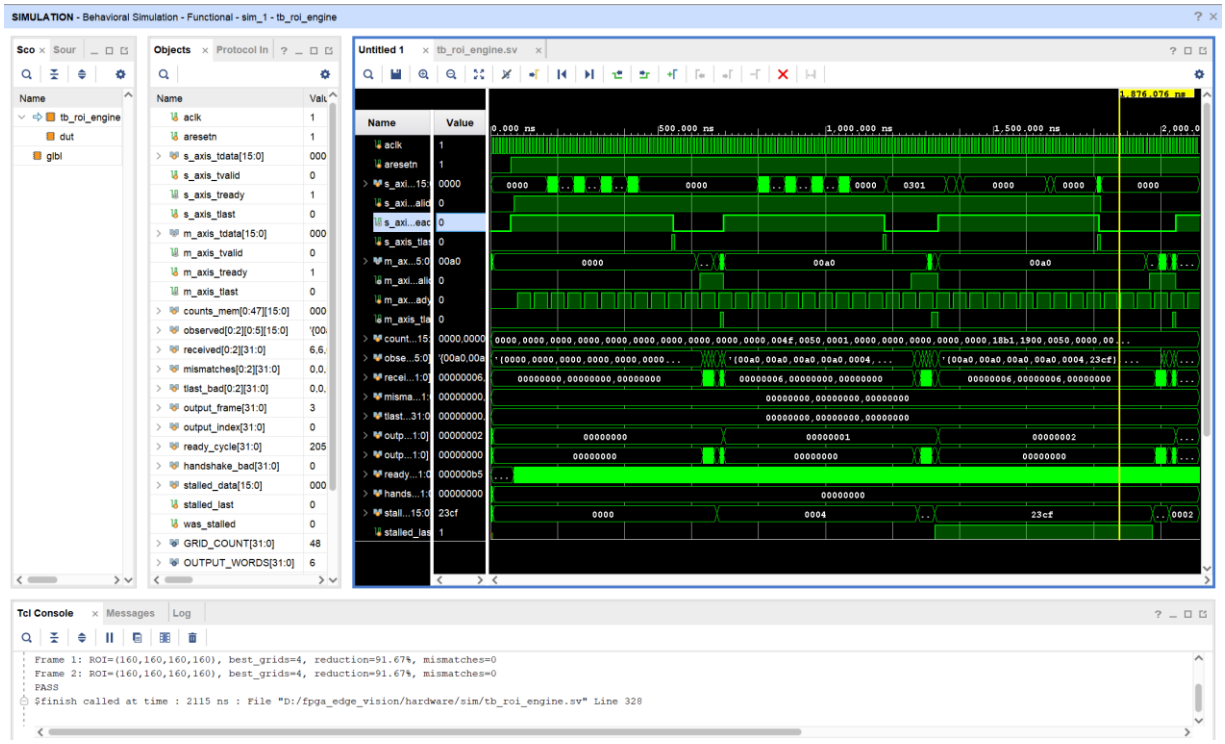
```
m_axis_tready <= ((ready_cycle % 5) != 0);
```

每 5 拍安排 1 拍暂停接收。测试平台在停顿期间检查 TVALID、TDATA 和 TLAST 是否保持稳定，并在有效传输时逐项核对输出字段，从区域筛选与接口传输两个方面检查模块行为。



ROI 搜索状态机与运算单元

模型仿真：



从 tcl 窗口可以得出：

```
Frame1:ROI=(160,160,160,160),best_grids=4,reduction=91.67%,mismatches=0
Frame2:ROI=(160,160,160,160),best_grids=4,reduction=91.67%,mismatches=0
PASS
```

仿真顺利通过。

2.2.3.5 双 DMA 与 AXI 系统互连电路

2.2.3.5.1 数据搬运与控制接口

（1）模块功能介绍

DMA 与互连电路负责把 DDR 中的灰度图转换为流式像素，并把边缘图、ROI 参数重新写回 DDR。本部分介绍的是 Vivado 中配置和连接的硬件通道，板端 Python 的摄像头读取、图像复制及网络处理另在软件部分说明。

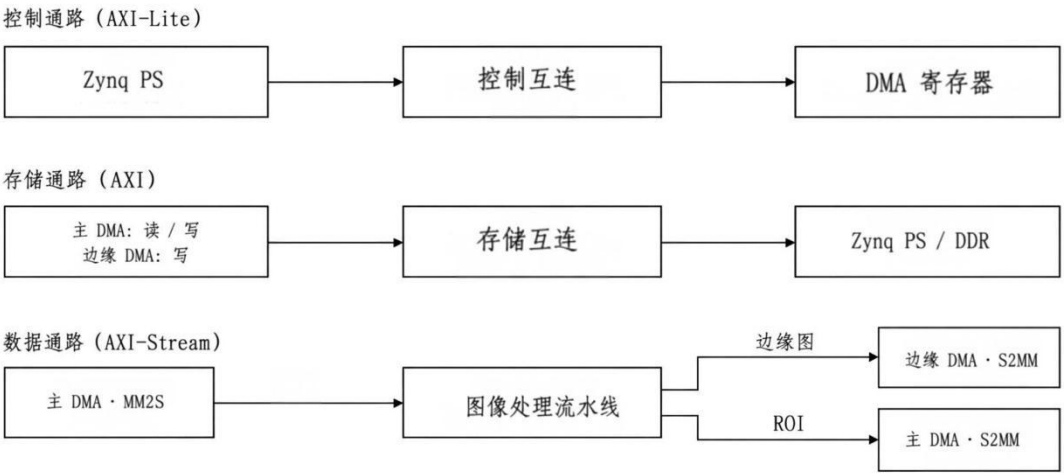
主 DMA axi_dma_0 同时启用 MM2S 和 S2MM，分别用于读取输入灰度图和接收 ROI。第二组 axi_dma_edge 仅启用 S2MM，接收二值边缘图。两组均关闭 Scatter-Gather，采用简单传输方式。

（2）模块接口设计

控制通路从PS的 M_AXI_GP0 出发,经过一入两出的axi_interconnect_ctrl,分别连接两组 DMA 的 S_AXI_LITE。PS 通过该接口访问 DMA 控制、地址、长度及状态寄存器。

存储通路包括主 DMA 的 M_AXI_MM2S、M_AXI_S2MM，以及边缘 DMA 的 M_AXI_S2MM，共三个存储访问主接口。它们接入三入一出的 axi_interconnect_mem，再连接 PS 的 64 位 S_AXI_HP0。HP0 将这些访问送往 PS 存储系统。

主 DMA 的流式发送宽度为 8 位，连接自定义模块 s_axis；其流式接收宽度为 16 位，连接 m_axis。边缘 DMA 的接收宽度为 8 位，连接 edge_m_axis。各 DMA 的内存侧宽度为 64 位，流侧与内存侧的位宽转换由 DMA 内部完成。



双 DMA、AXI 控制与 DDR 访问结构

处理开始前，需要为两条接收通路提供有效目标地址和接收长度，再启动灰度发送，使 PL 生成结果时具备接收条件。边缘图接收可与灰度发送同时进行，ROI 则在统计和搜索结束后输出。

ROI 面积缩减率只描述候选矩形面积，当前接收完整边缘图的通路仍需写入整帧数据。各 S2MM 接口接收数据包时，TDATA、TVALID、TREADY 与 TLAST 必须保持一致。ROI 最后一字段或边缘图末像素被暂停时，TLAST 必须保持到握手；提前撤销或遗漏结束标志，会破坏本工程的包结束约定。

2.2.3.2.5.2 工作时钟与复位分配

（1）统一时钟结构

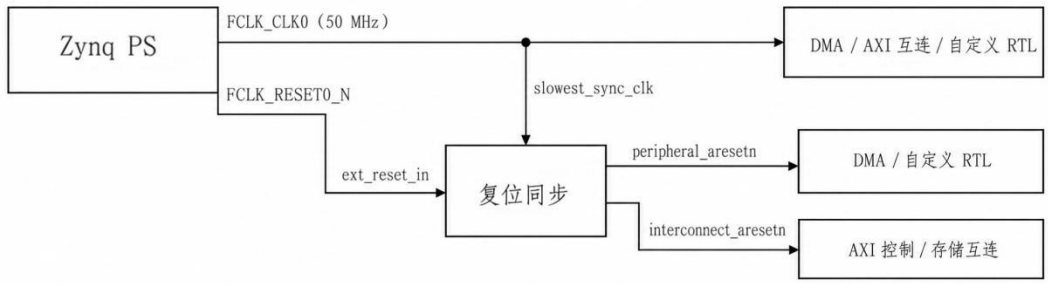
当前 PL 系统使用 PS 输出的 FCLK_CLK0，频率为 50 MHz。该时钟同时连接两组 DMA 的控制与数据时钟、两组 AXI 互连时钟、自定义 RTL 的 aclk，以及 PS 暴露的 GP0、HP0 接口时钟。

统一 PL 时钟并不表示 PS 内核、DDR 或板载 USB 等都运行在 50 MHz；这些部分的时钟和转换由相应控制器及器件负责。本工程使用的是已有 PS 外设和 DDR 控制器，不在自定义 RTL 内直接处理摄像头 ULPI、以太网 RGMII 或 DDR3 物理层时序。

（2）复位接口与状态初始化

FCLK_RESET0_N 接入 rst_ps7_0_50m 的 ext_reset_in，复位控制器以 FCLK_CLK0 为同步参考。peripheral_aresetn 分配给两组 DMA 的 axi_resetn 和自定义模块 aresetn，interconnect_aresetn 分配给两组 AXI 互连。未使用的低有效 aux_reset_in 接高电平，mb_debug_sys_rst 接低电平，dcm_locked 接高电平。

复位后，Sobel 清除流水有效位和坐标，网格统计清除累加器并返回接收状态，ROI 清除位图与区域寄存器，Broadcast 模块清除两路有效状态。系统由此丢弃尚未交付的旧数据，重新等待完整输入。



PL 统一时钟与复位分配

2.3 软件系统介绍

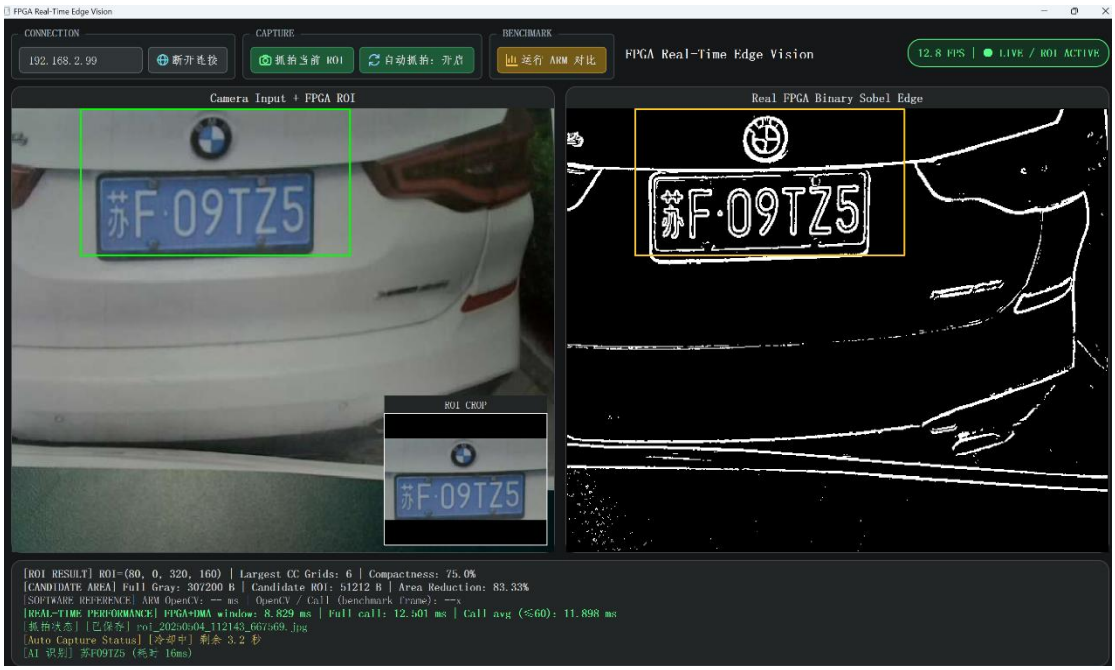
2.3.1 软件整体介绍

（含 PC 端或云端，结合关键图片）；

系统软件由 PYNQ 板端服务、PyQt5 客户端和电脑端识别服务组成。板端负责图像采集、硬件调用、抓拍与数据发送，客户端负责视频展示和操作控制，识别服务负责调用预训练模型完成车牌检测与识别。

系统运行时，板端从 USB 摄像头读取彩色图像，将对应灰度图通过 DMA 送入 PL，并通过两路接收通道获取 ROI 六字段和二值边缘图。原始彩色图像与边缘图经过编码后，通过 TCP 发送至电脑客户端，客户端绘制 ROI 框及局部预览，并显示处理耗时和筛选状态。

当用户手动抓拍或候选区域满足稳定条件时，板端裁剪并保存当前彩色 ROI，通过后台线程向电脑端发送 HTTP 识别请求。识别服务调用 HyperLPR3 模型，将车牌号码及耗时以 JSON 格式返回板端，再由板端通过控制连接更新客户端显示。



PC 端图片

2.3.2 软件各模块介绍

（根据总体框图，给出各模块的具体设计说明。从顶层到底层逐次给出各函数的流程图及其关键输入、输出变量）；

（1）板端主控与抓拍调度模块

该模块运行于 PYNQ 的 PS 端，负责初始化摄像头、FPGA 运行环境与网络服务，组织视频处理循环，并调度手动抓拍、自动抓拍及 ARM 同帧对比任务。控制接收线程将上位机指令放入队列，由主循环统一执行摄像头读取、DMA 调用和图像保存；车牌识别请求在后台线程中处理，主循环继续完成视频采集与发送。

核心函数：main()、serve_client()、save_roi()、complete_auto_capture()

函数名	功能
main()	管理硬件与网络，客户端连接
serve_client()	视频处理，调度指令，硬件处理和数据回传
valid_roi()	检查区域尺寸及坐标
save_roi()	裁剪彩色图，保存图片
encode_images()	将彩色图编码为 JPEG，边缘图编码为 PNG
AutoCapture.ready()	自动抓拍条件判断
AutoCapture.()	计算交并比
AutoCapture.saved()	更新抓拍成功状态，启动 4 秒冷却

板端主控与抓拍调度模块核心函数说明

关键输入： 摄像头对象 camera、指令队列 commands、FPGA 返回的 ROI 字段 fields 与边缘图 edge、后台识别结果。

关键输出： 视频数据包、运行状态、抓拍触发信号及保存成功后的文件路径 path。

运行流程： 读取控制指令 → 采集彩色图 frame → 转换灰度图 gray → 调用 FPGA 获取结果 → 按需执行 ARM 同帧对比 → 处理抓拍任务与识别结果 → 编码并发送视频及状态。

① 视频处理与任务调度

主循环读取摄像头的原始彩色帧，并转换为 640×480 灰度图，交由 DMA 运行模块送入 PL 端处理。硬件返回 ROI 字段与二值边缘图后，主循环依据控制指令和自动抓拍状态执行相应任务。ARM 对比使用当前同一灰度帧计算软件结果，并记录耗时；执行对比时重置自动抓拍的稳定计时，避免将基准测试造成的停顿计入稳定时间。

② 手动抓拍

收到手动抓拍指令后，主循环在当前 ROI 有效时，根据 fields 中的位置和尺寸，从原始彩色帧中裁剪对应区域，并调用 save_roi()保存图片。随后将文件路径交给异步识别客户端，获取车牌识别结果。手动抓拍由用户直接决定是否保存，不需要经过 1 秒稳定判定；即使后续未识别出车牌，手动保存的图片仍然保留。

③ 自动抓拍的空间一致性判定

自动模式开启后，AutoCapture.ready(fields)逐帧判断候选区域是否稳定。首次出现有效 ROI 时，将其记录为本轮参考框 R_{ref}，后续 ROI 记为 R_t，通过交并比衡量区域重叠程度：

$$\text{IoU}(R_t, R_{\text{ref}}) = \frac{|R_t \cap R_{\text{ref}}|}{|R_t \cup R_{\text{ref}}|}$$

为说明 IoU 与位置变化的关系，假设两个框的宽、高均为 w，h，仅在水平方向发生距离 d 的偏移，且 $0 \le d < w$，则：

$$\text{IoU}(d) = \frac{(w-d)h}{(w+d)h} = \frac{w-d}{w+d}$$

在这一条件下，偏移越大，$\text{IoU}(d)$越小。我们脚本程序采用的阈值为 0.6，因此仅考虑该 $\text{IoU}(d)$ 判据时：

$$\text{IoU} \ge 0.6 \quad \Leftrightarrow \quad d \le \frac{1-0.6}{1+0.6} w = \frac{w}{4}$$

宽度为 160 像素的等尺寸候选框，水平偏移 40 像素时，$IoU = 0.6$。

当 $\text{IoU}(R_t, R_{\text{ref}}) \ge 0.6$时，认为当前区域与参考区域一致。考虑到硬件网格化输出可能引起边框跳动，我们的程序还允许另一种匹配情况：两个框的中心在水平、垂直方向上的偏移均不超过 40 像素，且面积比处于 0.5～2 之间。

④ 1 秒稳定计时与异常处理

模块使用单调时钟 time.monotonic()计量时间，并由 elapsed 累计有效稳定时长。连续匹配参考框时，累加相邻两次处理的时间间隔；出现单帧异常时暂停累计并保留已有进度，异常间隔及恢复首帧的间隔均不计入。连续两帧异常则清零进度，重新等待或建立参考区域。

若相邻两次处理间隔超过 1 秒，也会重置计时，避免把程序停顿误判为目标稳定。累计有效稳定时间达到 1 秒后，模块输出触发许可，由主循环尝试提交自动识别任务。

⑤ 识别确认、保存与冷却

自动抓拍达到稳定条件后，主循环裁剪当前彩色 ROI，并将图像交给后台识别线程。此时先进行识别，不立即生成抓拍文件。只有返回结果满足三个条件，主循环才将对应图像保存为 auto_roi_文件；识别失败或超时则不保存。

a.任务状态转换

任务提交后，pending 标志阻止重复触发。任务处理结束后进入 4 秒冷却期，期间暂停稳定累计；冷却结束后重新等待目标。自动开关切换时，模块清除原有计时状态，并更新 generation 任务标记，使上层程序能够忽略失效的旧请求结果。

核心代码

```
auto_completed = complete_auto_capture(ai, automatic,
save_dir)
if not benchmark_pending and not auto_completed and
  automatic.ready(fields):
   x, y, w, h = fields[:4]
    if ai.send_candidate(frame[y:y + h, x:x + w],
      automatic.generation):
    automatic.submitted()
```



自动抓拍冷却时间显示图

b. 提交候选图

当目标稳定后，主循环直接从当前彩色帧的内存中裁出小图，提交给后台识别，提交后状态机挂起，暂停新的自动抓拍触发；同时生成一个 generation 批次号，用于过滤旧的识别结果。

```
if ai.send_candidate(frame[y:y + h, x:x + w],
  automatic.generation):
automatic.submitted()
```

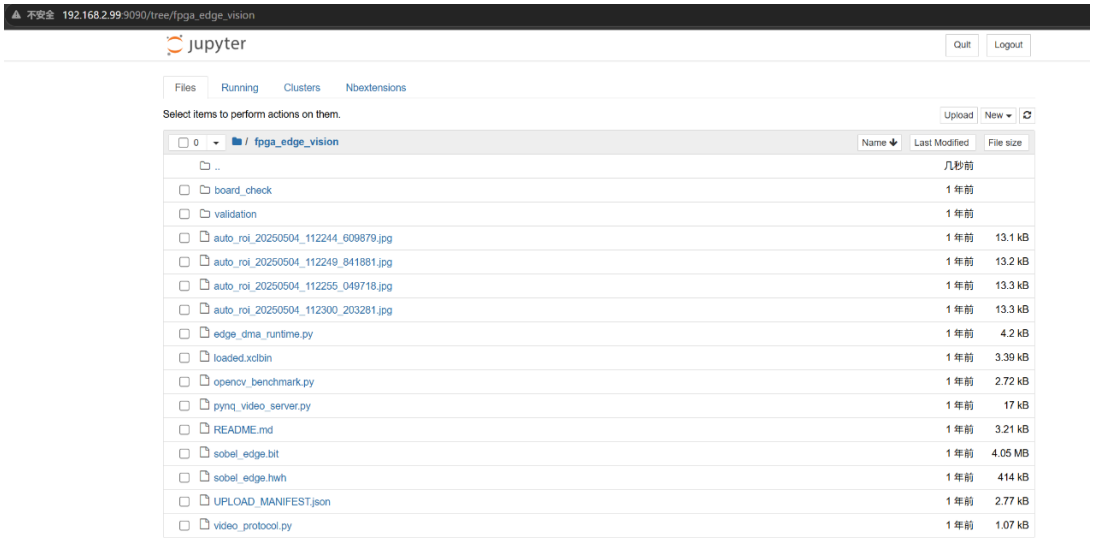
其中 submitted()如下：

```
def submitted(self):
    self.reset()
    self.pending = True
    self.message, self.kind = "[处理中]", "pending"
```

c.后台判别

后台线程收到 AI 服务端的返回结果后，当同时满足识别成功、车牌号非空以及服务端允许保存，后台回传图片数据，否则丢弃。

```
    plate = result.get("plate")
    if (result.get("status") == "success" and
isinstance(plate, str) and plate.strip()
            and (generation is None or
result.get("auto_save_allowed") is True)):
        elapsed = result.get("time_ms")
        timing = f" (耗时 {float(elapsed):.0f}ms)" if elapsed
is not None else ""
        message = f"[AI 识别] {plate.strip()}{timing}"
        kind = "saved"
        saved_jpeg = jpeg
```



后台保存图片（自动抓拍模式下）



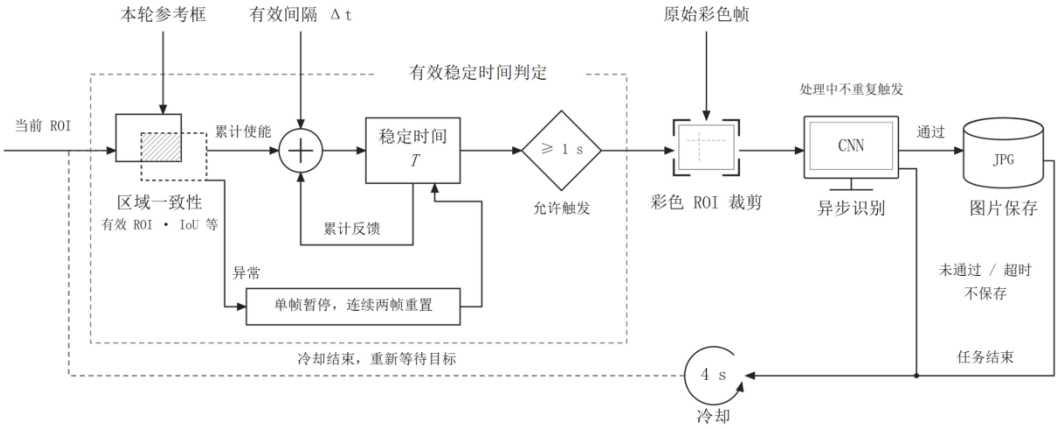
自动抓拍的后台车牌图片预览

⑧主循环保存

主循环从后台识别结果队列中非阻塞地取出结果，队列为空时直接返回。随后检查自动模式是否仍开启、generation 批次号是否有效。通过识别确认后，先写入临时文件，再替换为正式 JPEG 文件，保存内容与送去识别的图片一致。保存成功后，进入 4 秒冷却期。

```
def complete_auto_capture(sender, automatic, folder):
    # ……省略队列为空时的处理……
    result = sender.results.get_nowait()
    if not automatic.enabled or result["generation"] !=
automatic.generation:
        return False
```

```
    if result["kind"] != "saved":
        automatic.finish("等待目标", "idle")
        return True
    path = folder / ("auto_roi_" +
datetime.now().strftime("%Y%m%d_%H%M%S_%f.jpg"))
    temporary = path.with_suffix(".jpg.tmp")
    # ……省略文件写入失败时的代码……
    temporary.write_bytes(result["jpeg"])
    temporary.replace(path)
    automatic.saved(path)
    return True
```



自动抓拍稳定判定与调度原理示意图



自动抓拍状态变化与识别确认结果

（2）硬件驱动模块

该模块封装 Overlay 加载、DMA 缓冲区管理及数据收发。输入、ROI 和边缘图缓冲区在初始化时分配，后续逐帧复用。

核心函数： EdgeDmaRuntime.run_gray()

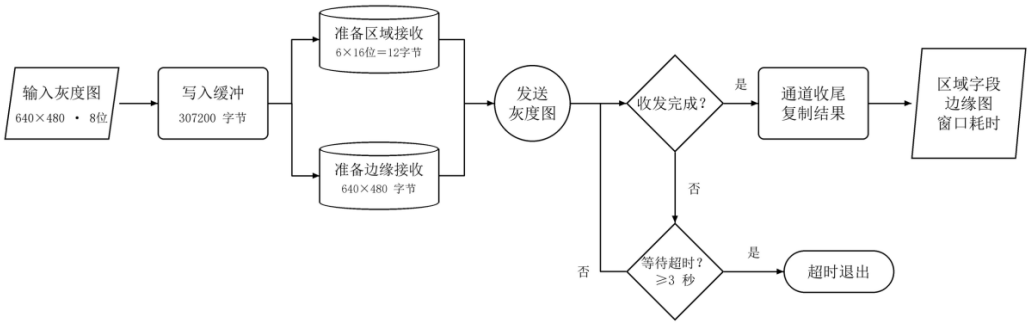
函数名	功能
init()	加载 Overlay，分配三块物理缓冲
run_gray()	发送灰度图，返回耗时
status()	DMA 通道的状态寄存器，返回状态
print_status()	打印 DMA 状态
close()	按传输完成状态保存或释放缓存

硬件驱动模块核心函数

关键输入： gray，640×480、uint8 格式的灰度图

关键输出： fields、edge、dma_ms，分别表示 ROI 六字段、边缘图副本及 DMA 时间窗口

运行流程： 写入输入缓冲 → 启动两路接收 → 启动灰度图发送 → 轮询等待完成 → 同步缓存并检查接收长度 → 复制结果并返回



核心代码

```
self.in_buf[:] = gray.reshape(-1)
started = time.perf_counter()
self._safe_to_free = False
self.dma_roi.recvchannel.transfer(self.roi_buf)
self.dma_edge.recvchannel.transfer(self.edge_buf)
self.dma_roi.sendchannel.transfer(self.in_buf)
deadline = time.monotonic() + self.timeout_s

while not all(channel.idle for channel in self._channels):
    if time.monotonic() >= deadline:
        self.print_status("DMA 超时")
           raise TimeoutError(f"双 DMA 传输超过
{self.timeout_s:.1f
}秒")
    time.sleep(0.001)

for channel in self._channels:
    channel.wait()
self._safe_to_free = True
```

```
elapsed_ms = (time.perf_counter() - started) * 1000
# 复制结果，避免下一帧传输覆盖
roi = self.roi_buf.tolist()
edge = np.array(self.edge_buf).reshape(HEIGHT, WIDTH)
return roi, edge, elapsed_ms
```

① 输入缓冲与复用

gray.reshape(-1)将二维灰度图数组，再复制到物理连续的输入缓冲区 in_buf。初始化分配输入、ROI 和边缘图缓冲区，后续逐帧复用，减少重复申请和释放内存开销。

② 两路接收与一路发送

程序先启动 ROI 字段接收和边缘图接收，再发送灰度图。三个通道分别承担以下任务：

- dma_roi.recvchannel：接收 ROI 六字段
- dma_edge.recvchannel：接收二值边缘图
- dma_roi.sendchannel：发送输入灰度图

③ 完成等待与异常保护

程序轮询_channels 中三个通道的 idle 状态，全部空闲则结束轮询，随后调用 wait()完成等待与驱动收尾。

timeout_s 默认设为 3 秒，限制轮询等待阶段。超过期限时，程序打印 DMA 状态。此时_safe_to_free 仍为 False，后续清理会保留缓冲区，避免在 DMA 尚未确认结束时释放物理内存。

④ 结果复制与耗时范围

传输确认完成后，程序将 ROI 缓冲区转换为列表，将边缘图复制到独立数组并恢复二维形状，避免下一帧 DMA 写入覆盖当前结果。

（3）软件基准对比模块

该模块在 ARM 处理器上使用 OpenCV 实现与硬件一致的处理规则，用于核对 ROI 结果并比较前处理耗时。参考算法仅在收到 BENCHMARK 指令时执行。

核心函数： opencv_roi_pipeline()、result_fields()

关键输入： 当前测试帧的灰度图 gray

关键输出： 包含 ROI 与边缘统计的结果字典；整理后六字段列表 arm_fields

运行流程： Sobel 梯度计算 → 幅值求和及饱和处理 → 边界清零 → 阈值 80 二值化 → 8×6 网格统计 → 判断计数大于 768 的活跃网格 → 四连通分析 → 计算轴对齐包围框和面积缩减率。

主程序将同一张灰度图分别交给 OpenCV 和 FPGA，记录 arm_ms 与 call_ms，比较六字段是否一致。

核心代码：

```
    if benchmark_pending:
        from opencv_benchmark import opencv_roi_pipeline,
result_fields
        automatic.reset()
        started = time.perf_counter()
        arm_fields = result_fields(opencv_roi_pipeline(gray))
        arm_ms = (time.perf_counter() - started) * 1000
        started = time.perf_counter()
        fields, edge, dma_ms = runtime.run_gray(gray)
        call_ms = (time.perf_counter() - started) * 1000
        fields = list(map(int, fields))
        durations.append(call_ms)
    if benchmark_pending:
        benchmark=dict(frame_id=
frame_id,arm_ms=arm_ms,fpga_ms=call_ms,
        match=arm_fields == fields)
    automatic.reset()
```

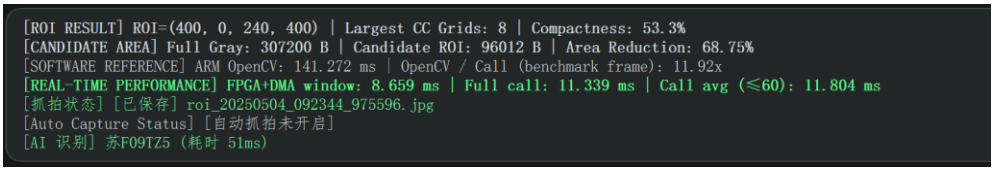
同一灰度帧上的软硬件对比，主循环生成当前帧的灰度图 gray，再将 gray 依次传给 OpenCV 和硬件驱动。因此两条路径处理的是相同图像内容，这样减少摄像头场景变化对结果比较的影响。

为保证对比的公平性，主循环将当前帧灰度图 gray 作为唯一输入，依次送入 OpenCV 参考算法与 FPGA 硬件驱动。

①耗时统计：使用 time.perf_counter() 分别记录耗时。arm_ms 为 OpenCV 完整计算及结果整理时间；fpga_ms 记录硬件完整调用耗时（call_ms），同时保留驱动内部的 DMA 窄窗口耗时（dma_ms）。主循环维护滑动窗口，计算平均耗时。

②结果比对：将硬件结果转换为整型后，与 ARM 结果逐项比对坐标、宽高、网格数等 6 个 ROI 字段。全部一致时 match 为真。比对结果绑定 frame_id 发送至网页端显示。

③状态隔离：测试前后调用 automatic.reset() 冻结自动抓拍计时器，防止因对比测试造成的耗时停顿被误判为目标稳定，干扰自动抓拍状态机。



软硬件对比示例

（4）TCP 视频与控制通信模块

该模块为视频和控制消息建立独立连接。视频通道使用 8080 端口，发送帧号、ROI 坐标、DMA 耗时及 JPEG、PNG 图像；控制通道使用 8081 端口，传输按钮指令、抓拍状态和识别结果等信息。

核心函数：

函数名	功能
socket.socket()	创建 TCP 套接字，用于端口连接
connect()	连接板端 8080、8081 端口
bind()	将监听套接字绑定指定地址和端口
listen()	开启监听，等待连接
accept()	接受连接，获得套接字
recv()	读取数据，接收指令、运行状态
sendall()	发送指定字节，异常报错
shutdown()	停止套接字收发，结束通信
close()	关闭套接字并退出

TCP 基础接口函数说明

函数名	功能
pack_frame()	自定义包头，拼接彩色图与边缘图数据
recv_exact()	接收确定长度数据

recv_frame()	解析自定义包头，接收图像数据
QImage.fromData()	将 JPEG、PNG 字节解码为图像
ControlLink._init()_	启动控制接收线程
ControlLink._receive()	解析 JSON 消息，指令送入队列
ControlLink.send()	将运行状态编码为 JSON 并发送
ControlLink.close()	关闭线程并退出

视频与控制处理函数说明

关键输入： frame_id、ROI 坐标、dma_ms、两张图像的编码数据，以及控制指令

关键输出： 完整视频数据包、解析后的图像数据及指令队列

视频流程： 编码图像 → 打包 24 字节包头 → 发送图像数据 → 客户端按长度读取 → 解包。

控制流程： 客户端发送 JSON 指令 → 板端接收并入队 → 主循环执行 → 返回状态 JSON。

代码示例

```
def recv_exact(sock, length):
data = bytearray()
   while len(data) < length:
     block = sock.recv(min(length - len(data), 65536))
    if not block:
     raise ConnectionError("连接已断开")
data.extend(block)
return bytes(data)
```

length 为本次需要接收的字节数，data 存有已经收到的数据。程序每次根据剩余长度继续读取，累计长度满足要求时再返回。该函数用于读取 24 字节自定义包头，再根据包头数据，依次读取 JPEG 和 PNG 数据。其中，包头解析由 HEADER.unpack()完成，recv_exact()只负责接收指定长度的字节。

（5）异步识别请求模块

该模块将已保存的候选彩色 ROI 发送至电脑端识别服务。HTTP 请求在后台线程中执行，避免主循环同步等待识别响应。

核心函数： AsyncPlateSender.send_async()、_send()、status()

关键输入： 抓拍图像路径 path 和识别服务地址 url

关键输出： 识别提示 message、状态类型 kind 及忙标志 busy

运行流程： 检查忙标志 → 提交后台任务 → 读取 JPEG 文件 → 发送 HTTP POST 请求 → 解析 JSON → 加锁更新状态 → 清除忙标志。

请求超时设为 3 秒。已有请求未完成时，新的识别请求不重复提交；已保存的抓拍图像仍然保留。

```
    核心代码
    def _send(self, path):
        message, kind = "[AI 识别] 识别失败/超时", "error"
        try:
            request= urllib.request
    .Request(self.url, data=path.read_bytes(),
        headers={"Content-Type": "image/jpeg"},
method="POST")
            with self.opener.open(request,timeout=3.0) as
response:
        result = json.loads(response.read().decode("utf-8"))
        if result.get("status")=="success" and
result.get("plate"):
                message = f'[AI 识别]
    {result["plate"]}  (耗时 这
{float(result["time_ms"]):.0f}ms)'
                kind = "saved"
        except Exception:
            pass
```

后台线程读取本地保存的 JPEG 图片，通过 HTTP POST 发送给电脑，并强制设置了 3 秒超时，防止网络卡死。

如果电脑回传“成功”且带有车牌号，就记录成功提示；如果超时或报错，静默记为失败。识别任务结束时，_send()通过 finally 清除忙标志，因此网络异常或识别失败也能继续接受后续请求。

（6）PyQt5 显示与交互模块

客户端采用后台线程接收数据，GUI 主线程负责绘制界面。视频线程将接收到的 JPEG 和 PNG 解码为 QImage，主线程获取最新图像后转换为 QPixmap，更新原图、边缘图、ROI 框和局部预览。

核心类与函数：VideoThread、ControlThread、Dashboard.receive_frame()、show_frame()及 update_metrics()

关键输入： 图像包 packet、状态数据 status 及用户按钮操作

关键输出： 双画面、性能指标、抓拍和识别提示，以及发往板端的控制指令

显示流程： 后台接收 → 图像解码 → 发布最新帧 → GUI 获取图像 → 绘制 ROI 与局部预览 → 更新指标。

交互流程： 用户点击按钮 → 指令进入队列 → 控制线程发送 → 接收板端反馈 → 更新界面状态。

① 最新帧覆盖与事件合并

核心代码

```
def publish(self, packet):
    with self.lock:
        self.latest = packet
        notify = not self.notice_pending
        self.notice_pending = True
    if notify:
        self.available.emit()
def take_latest(self):
    with self.lock:
        packet, self.latest = self.latest, None
        self.notice_pending = False
    return packet
```

latest 缓存仅保留最新待显示帧，避免高频图像数据在接收队列中积压。notice_pending 标志位用于合并刷新通知，若 GUI 未处理上条通知，不会重复发射 available 信号避免卡顿。

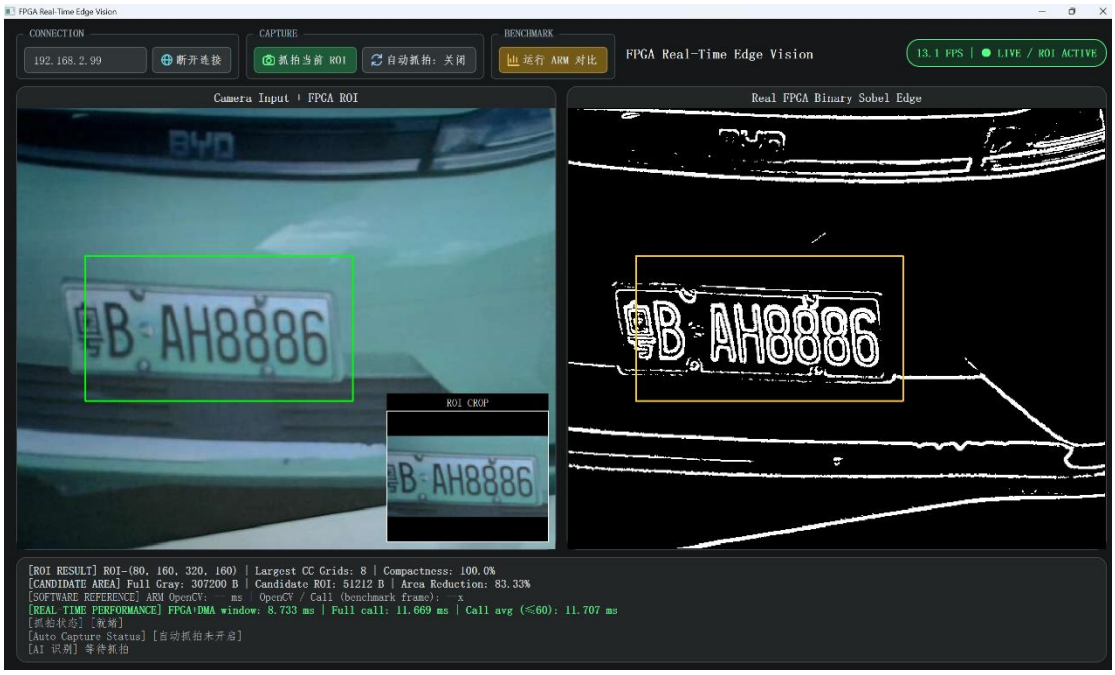
② GUI 主线程画面更新

```
    def receive_frame(self):
        packet = self.video.take_latest()
        if self.running and packet is not None:
            color, edge = packet.pop("color"),
packet.pop("edge")
            self.frames_ready.emit(QPixmap.fromImage(color),
QPixmap.fromImage(edge), packet)
    def show_frame(self, color, edge, packet):
        self.fps_frames += 1
        self.current = packet
        self.left.set_frame(color, packet["roi"])
    self.right.set_frame(edge, packet["roi"])
```

画面显示：后台线程负责解码成原始图像，主线程负责把解码后的图转成 UI 能显示的图片格式，并刷新左右双画面。

画框：因为界面窗口可能会被拉伸，控件会对原图和框坐标进行同比例缩放，确保框始终对准目标。

局部预览：右下角放大的小图，是电脑直接在大图上“裁”下来放大的。



PyQt5 客户端双画面显示与交互运行界面

（7）电脑端本地识别服务模块

该模块使用 Flask 提供 HTTP 接口，通过 ONNX Runtime 运行 HyperLPR3 预训练模型。模型在服务启动时加载一次，后续请求复用全局对象 MODEL。

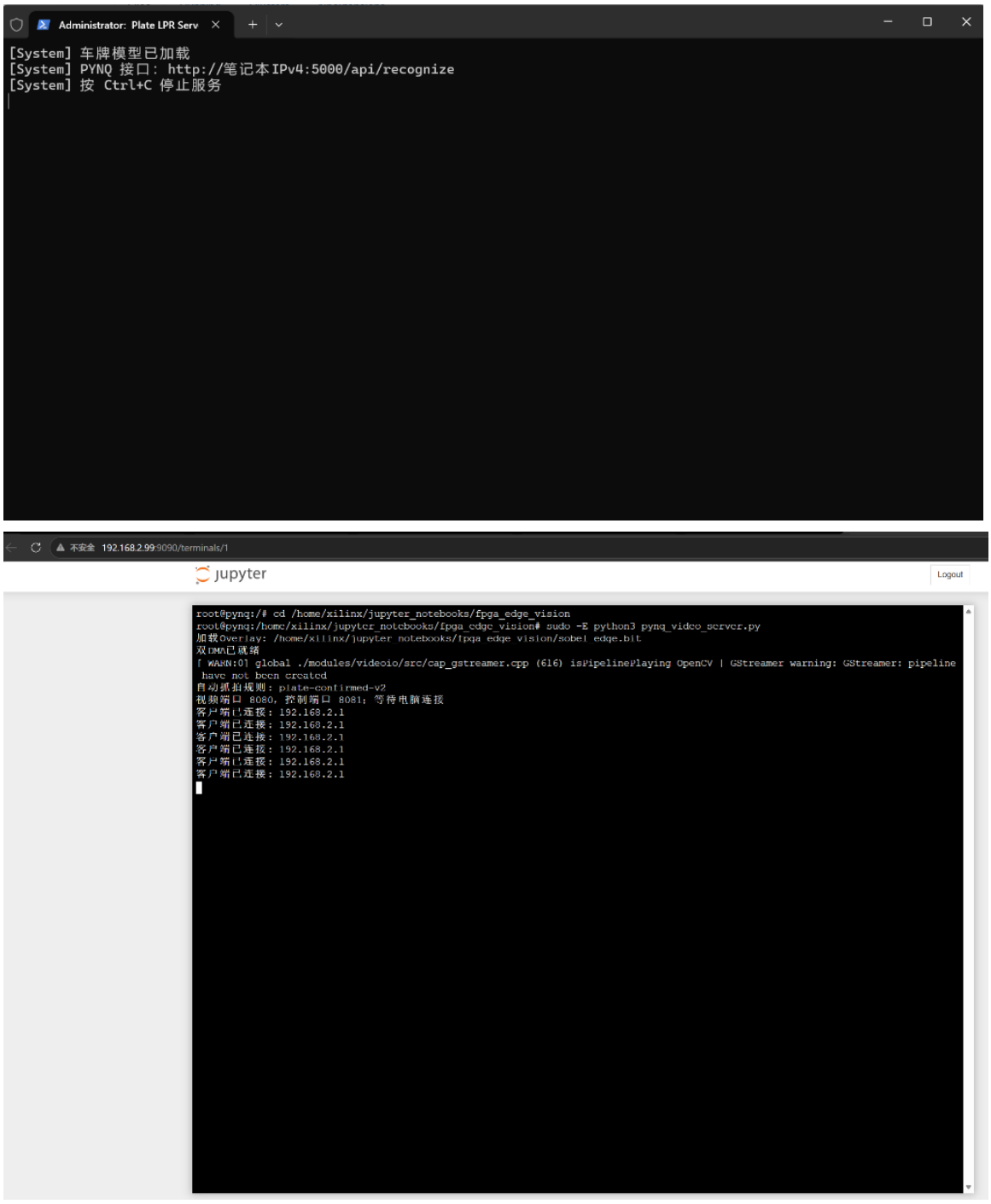
核心函数： load_model()、api_recognize()

关键输入： HTTP 请求体中的 JPEG 字节数据，解码后得到彩色图像 image

关键输出： 包含 status、plate 和 time_ms 的 JSON 响应

运行流程： 接收 JPEG 字节 → 解码彩色图像 → 调用模型 → 选择最高置信度车牌结果 → 封装 JSON → 返回板端。

没有检测到车牌时返回失败状态。time_ms 记录模型调用及结果选择耗时，不包含图像解码与 HTTP 传输时间。



电脑端识别服务启动与 HTTP 识别结果

第三部分　完成情况及性能参数

阐述最终实现的成果（图文结合，实物照片为主）

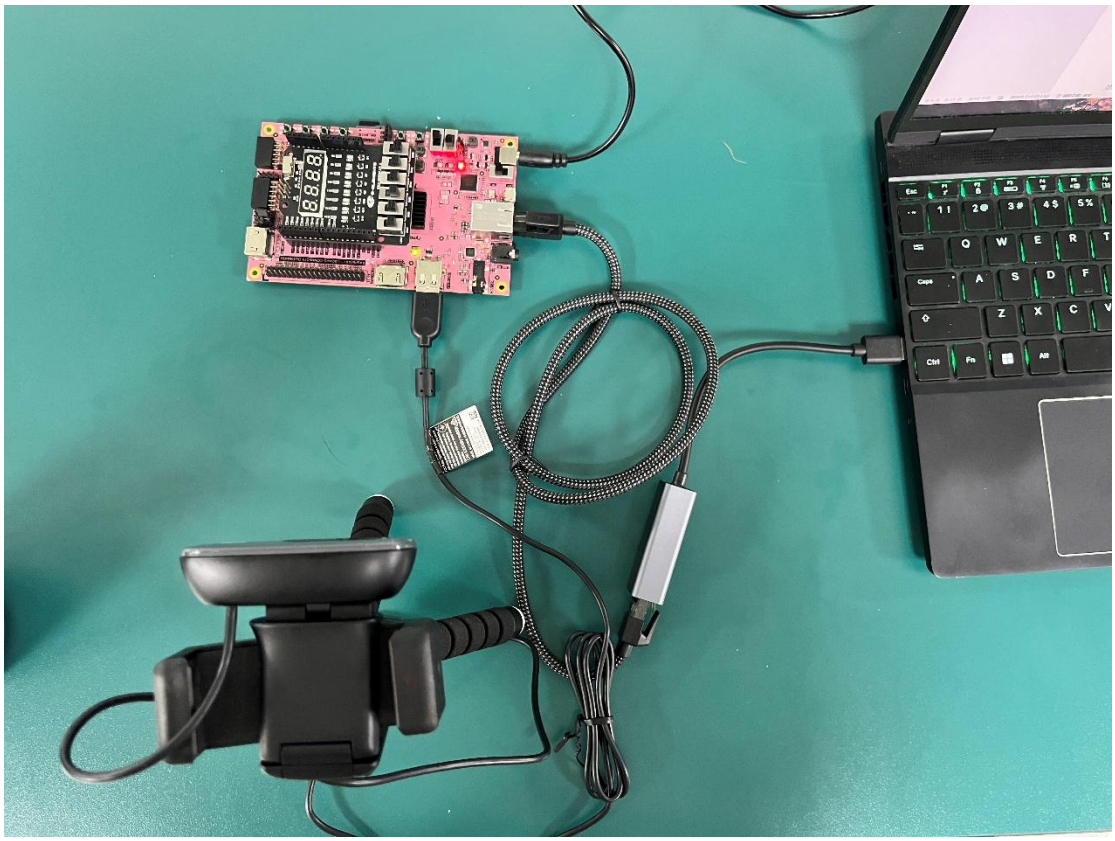
3.1　整体介绍

（整个系统实物的正面、斜 45°全局性照片）

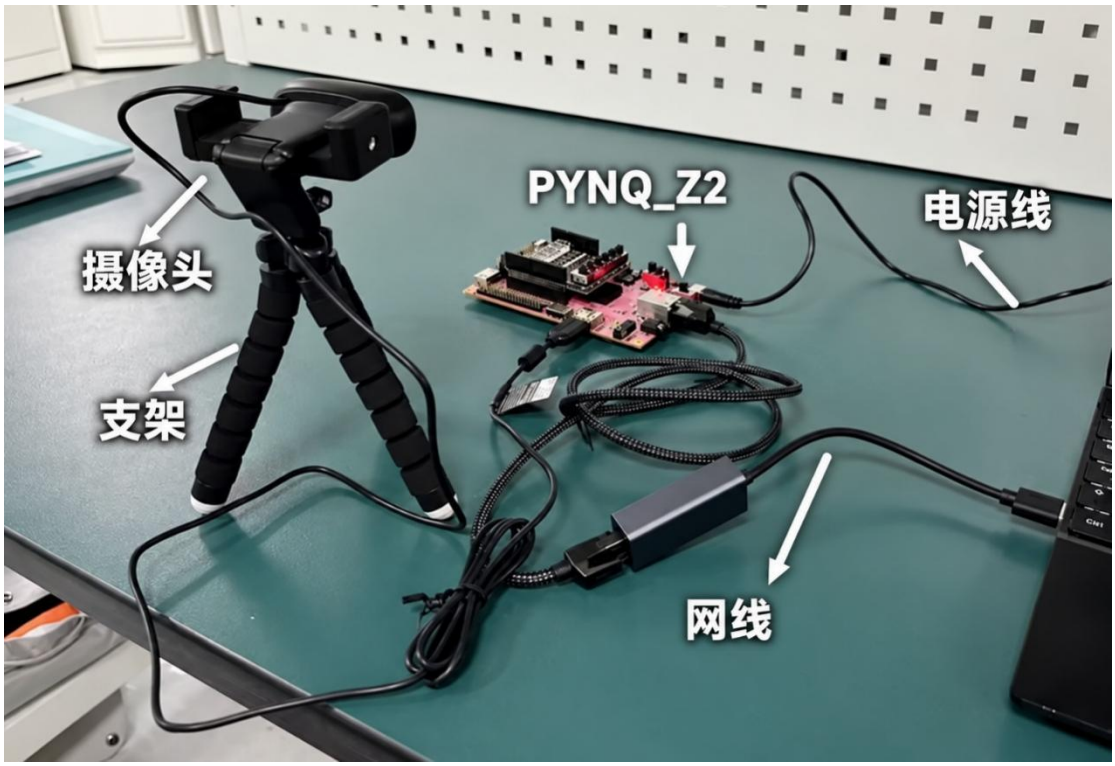
本系统是一套基于 PYNQ-Z2 的软硬件协同车牌识别系统，主要由 USB 摄像头、PYNQ-Z2 开发板和电脑组成。如图所示，摄像头安装在支架上，通过 USB 连接开发板；开发板独立供电，通过网线与电脑通信。

运行时，开发板的 ARM 端采集摄像头图像，将灰度图像通过 DMA 送入 FPGA。FPGA 完成 Sobel 边缘提取、网格统计和候选区域筛选，并返回边缘图与区域坐标。ARM 根据坐标裁剪原始彩色图像，将候选区域发送到电脑，由 HyperLPR3 模型完成车牌检测与字符识别。

电脑端使用 PyQt5 界面展示原始画面、边缘图、候选区域和识别结果，同时支持手动抓拍、稳定目标自动抓拍及处理耗时统计，实现从图像采集、硬件预处理到车牌识别和结果显示的完整流程。



系统俯视图



系统 45° 拍摄图

3.2 工程成果

（分硬件实物、软件界面等设计结果）

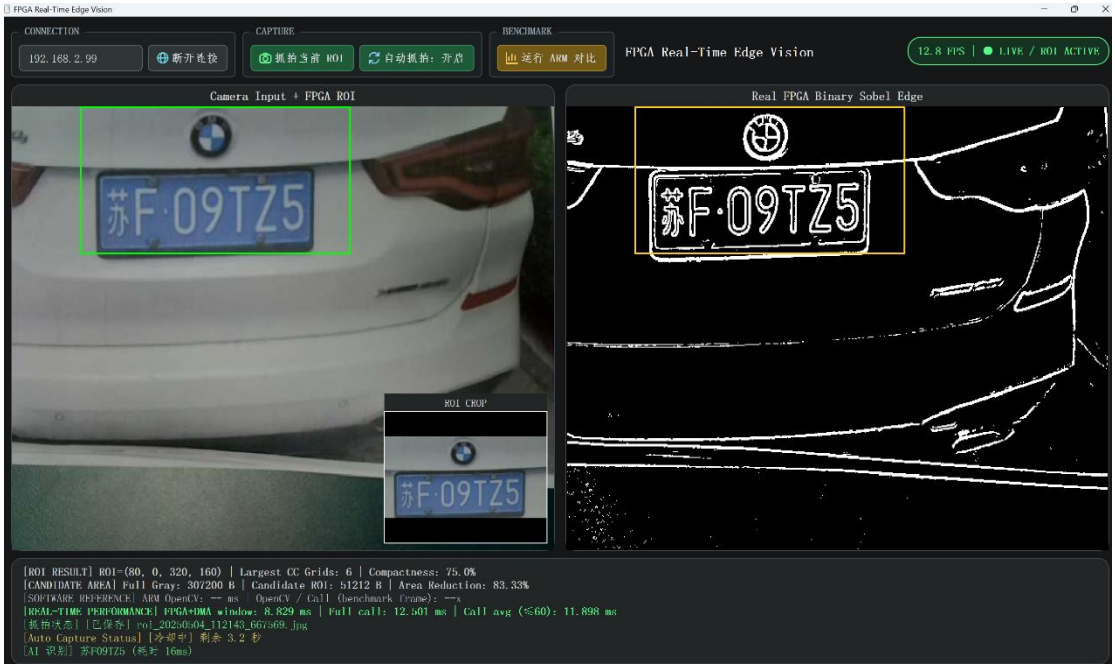
3.2.1 机械成果；（实物照片）

如上图所示。

3.2.2 电路成果；（实物照片）

如上图所示。

3.2.3 软件成果；（界面照片）



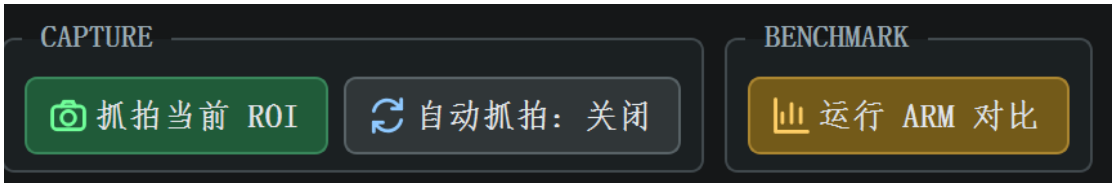
PyQt5 客户端界面

3.3 特性成果

（逐个展示功能、性能参数等量化指标）（可加重要仪器测试或现场照片）；

3.3.1 **功能一：抓拍当前 ROI 区域**

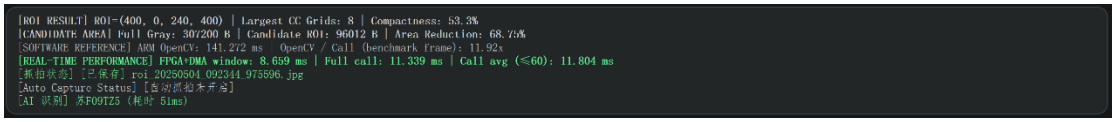
该功能用于手动保存当前画面中的候选区域。点击“抓拍当前 ROI”按钮后，系统根据 FPGA 返回的区域坐标，从摄像头采集的原始彩色图像中裁剪 ROI，并将图片保存到开发板。随后，板端将抓拍图像发送至电脑端的车牌识别服务，由模型完成车牌检测与字符识别。处理结束后，界面显示图片保存状态、车牌号码和识别耗时，方便用户查看当前目标的识别结果，也为后续测试保留图像样本。



三个主要功能按钮



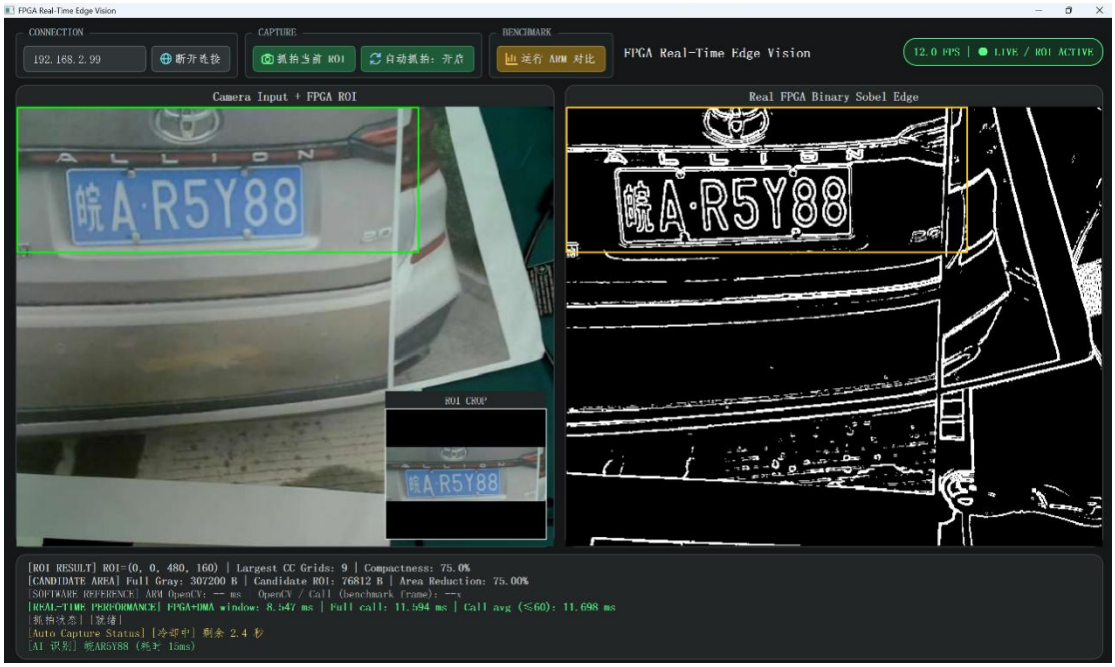
抓拍当前 ROI 左边区域



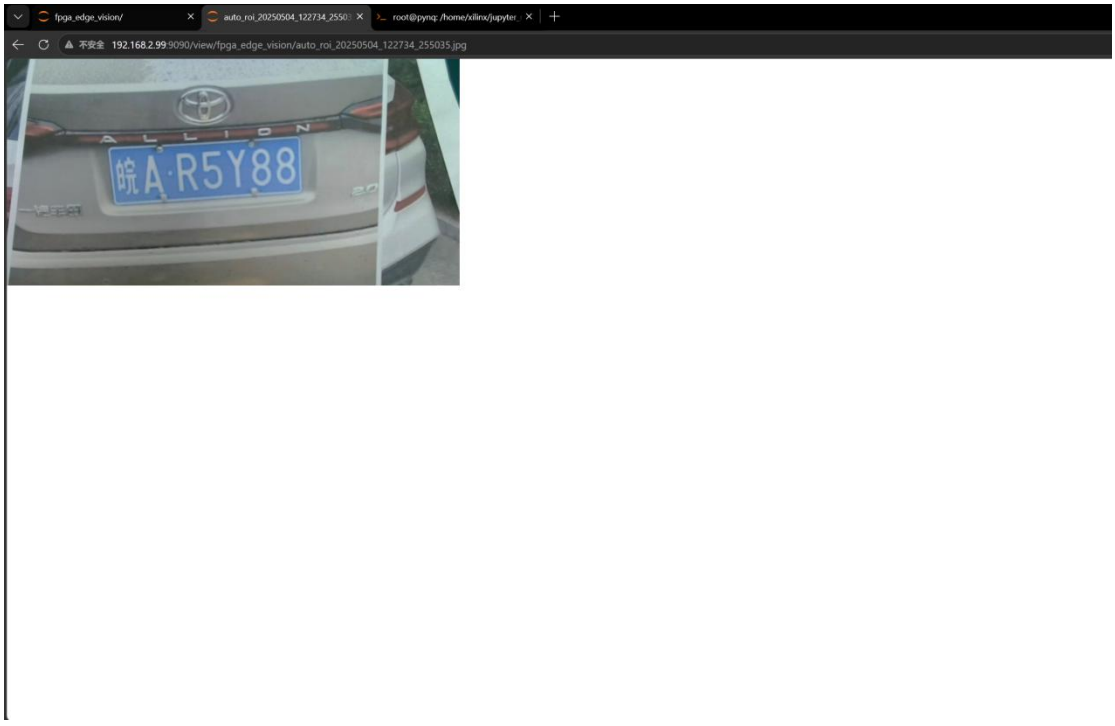
成功抓拍结果显示

3.3.2 功能二：自动抓拍

该功能用于连续运行过程中的自动采集。开启后，系统持续跟踪候选区域的位置和大小，判断目标是否保持稳定；当区域满足稳定条件时，自动完成彩色 ROI 裁剪、图片保存和识别请求。系统设置了抓拍间隔，限制短时间内的连续触发，减少画面抖动和目标移动带来的无效抓拍。用户可以在界面查看自动抓拍的开启状态、保存提示和识别结果，适合固定摄像头下的连续演示与图像采集。

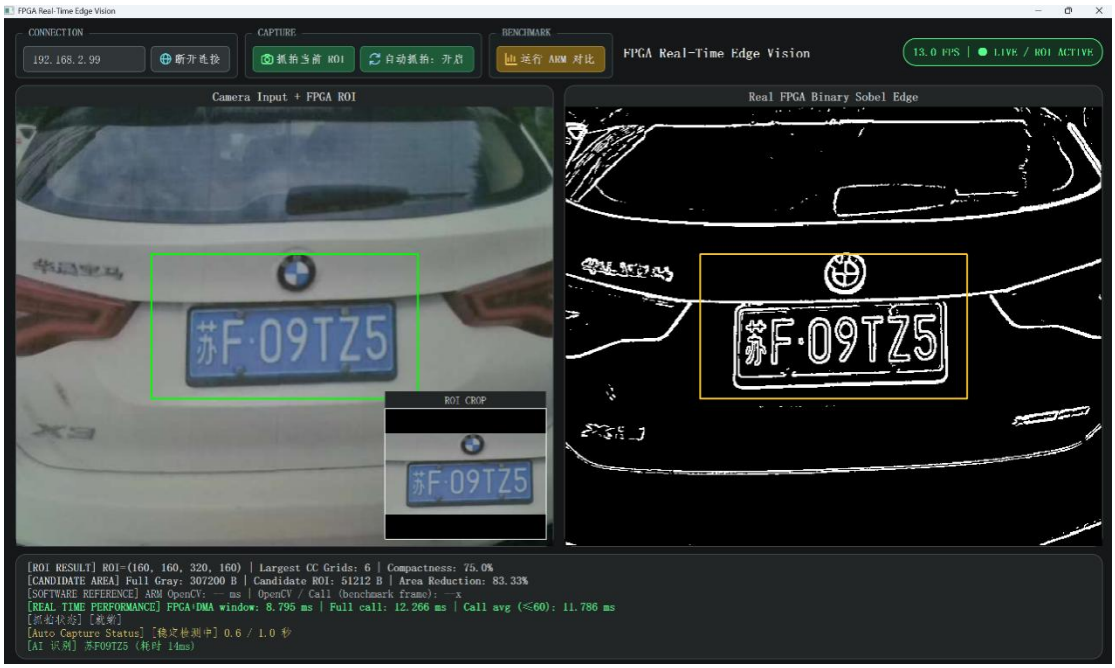


开启自动抓拍（冷却中）



查看自动抓拍保存的图片

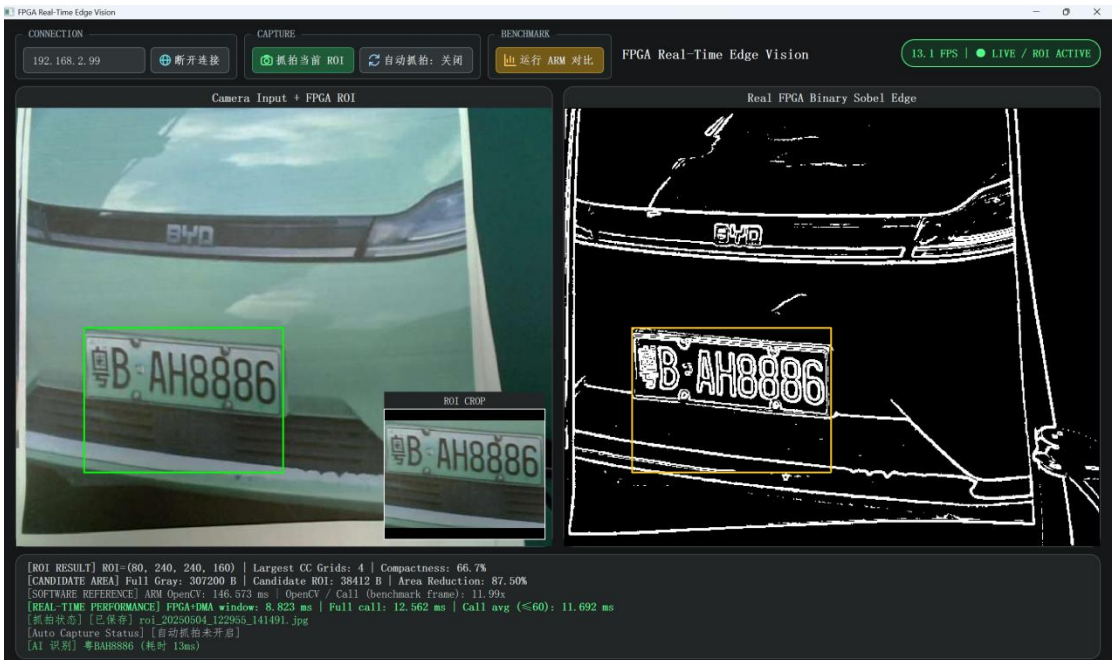
、



自动抓拍稳定检测中

3.3.3 功能三：运行 ARM 对比

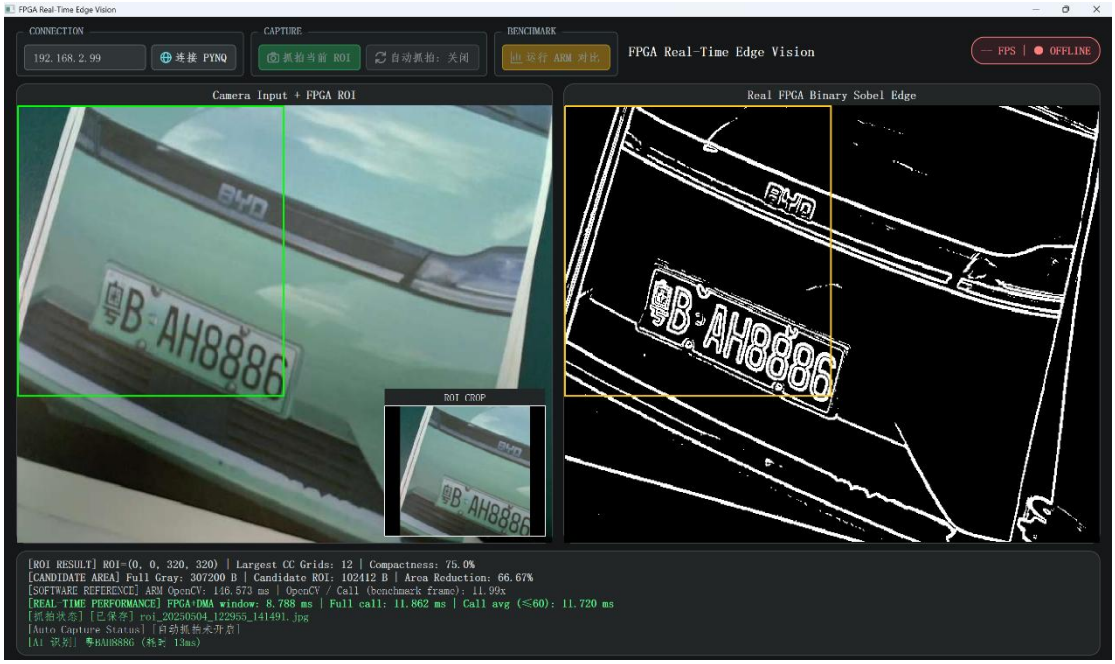
该功能用于验证 FPGA 处理结果，并观察软硬件两种处理方式的性能差异。触发后，系统对同一帧图像运行 ARM 端 OpenCV 参考算法，将生成的边缘图和候选区域坐标与 FPGA 输出进行核对。界面展示 ARM 参考算法耗时、FPGA 完整调用耗时，并在结果一致时给出加速比。其中，FPGA 完整调用包含数据复制、DMA 传输及硬件处理等开销，可用于评估图像预处理与候选区域筛选在实际调用中的加速效果。



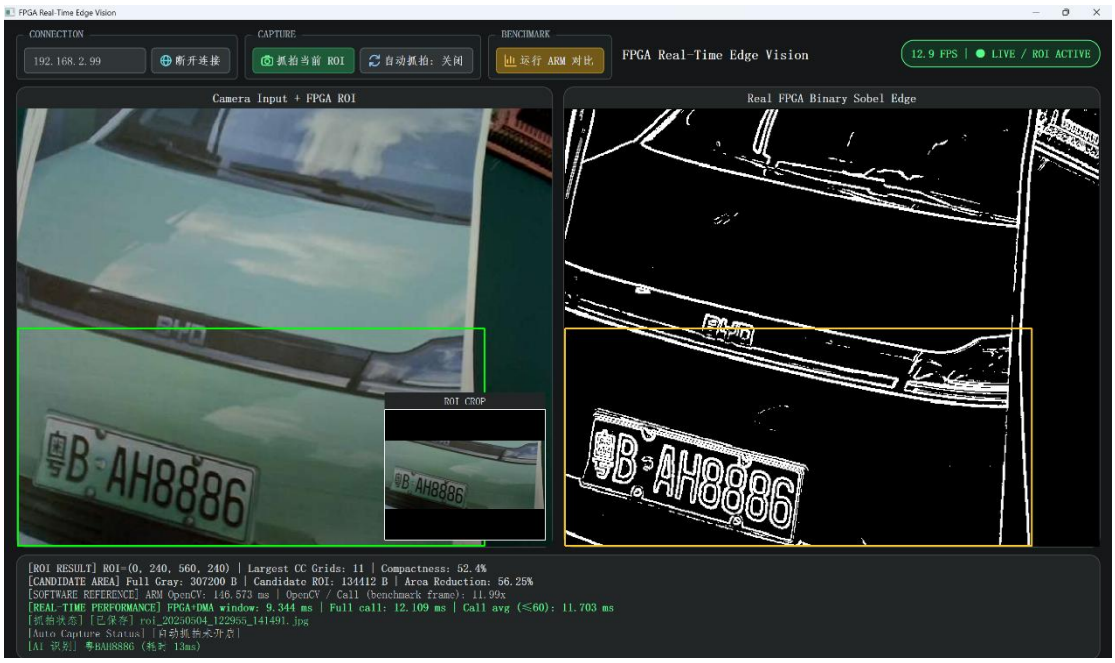
运行 ARM 对比结果图（下方展示运行处理时间）

3.3.4 功能四：通过按钮连接 PYNQ-Z2

该功能用于建立电脑界面与开发板之间的通信。使用前，需要确保电脑与 PYNQ-Z2 网络互通，并启动板端视频服务；随后在界面输入开发板的 IP 地址，点击“连接 PYNQ-Z2”。连接成功后，才能完成上面所示的三个功能。



断开 PYNQ 连接图（右上角 OFFLINE）



正常连接图（右上角显示 LIVE/ROI ACTIVE）

第四部分　总结

4.1 可扩展之处

300 字内；

后续可补充不同光照、拍摄距离和车辆移动状态下的测试，改进候选区域筛选，提高识别稳定性，并探索多目标处理。

抓拍功能可加入图像清晰度判断和重复车牌过滤，减少无效保存。同时，将抓拍图片、车牌号码和时间关联记录，增加历史查询与数据导出功能。

应用上，可尝试车辆出入记录、小型停车场管理等场景，通过实际使用继续完善系统，毕竟现在实验的环境过于简化。目前识别模型运行在电脑端，后续可研究模型轻量化，结合 PYNQ-Z2 的资源评估板端部署的可行性，逐步减少对电脑的依赖，把 CNN 模型部署到板子上。

4.2　心得体会

1000 字内，可包括研发和制作细节；

这次车牌识别系统的研发，是一次把课堂知识落实到实际工程中的过程。最初做这个项目时，我们只是想搭建一个能够完成图像采集、FPGA 处理和上位机显示的系统，当时并没有确定要做车牌识别，而且现在车牌识别确实太普遍了不是很创新。前期主要在解决摄像头怎么接入、图像怎么传到 FPGA、处理结

果怎么显示等问题，这一部分大概花了好几周吧。等这些部分逐渐跑通后我们才发现：系统虽然能展示处理效果，但还缺少一个比较明确的应用场景。

后来在测试过程中，从二维码、铭牌到拍环境，我们都试过拍摄，偶然发现数字识别的效果比较好，就开始考虑能不能围绕数字和字符找一个实际用途。车牌识别因此进入视野：它有比较明确的使用场景，也能把图像采集、候选区域筛选和字符识别串起来。项目后面的方向就是这样逐渐确定下来的，因为这个我们还改了下项目简介，感觉改之后确实比之前更加明确了。

方向确定后，我们逐步增加了自动抓拍和手动抓拍功能。自动抓拍需要判断候选区域是否稳定，满足条件后再保存图像，同时通过抓拍间隔减少连续触发。手动抓拍则方便我随时保存当前区域，检查裁剪结果。做这些功能时，我们开始关心操作是否方便、保存状态是否清楚，以及抓拍的图片能不能用于后续识别。

有了抓拍功能后，我们再接入 CNN 模型，由电脑端完成车牌检测和字符识别，开发板负责图像采集、预处理和候选区域裁剪。到这一步，采集、处理、抓拍和识别才形成完整的流程。我们也发现，接入模型需要处理好图像发送、结果返回和界面更新，让每一步都能衔接起来。

完善系统时，环境部署也花了不少精力。更换项目目录后，出现过找不到指定 Python 版本、缺少 PyQt5，以及仍然使用旧目录环境等问题。这些经历让我们开始认真核对解释器路径、依赖和启动顺序。后来整理文档时，我们也更在意别人能否照着步骤运行起来，仔细写好每个部分的 readme，确保把自己遇到的问题提前交代清楚。

回头看，这个项目的方向是在制作和测试中慢慢找到的。虽然说这个项目不是很创新，但起码我们投了挺多精力进去，然后也发现什么神经网络其实并非什么不可触及的东西，不过要自己训练还是很有难度且耗时间的，所以我们用的是开源模型。从搭建基础系统，到找到应用场景，再到补充抓拍和识别功能，每一步都让我们更清楚接下来需要解决什么。我们最大的收获，是学会根据实际效果调整方案，让系统真正用起来。无论结果如何，我们都尽力了。

第五部分　参考文献

按照标准格式，限 20 篇以内。

[1]赵箐祥,廖荣. 基于动态 ROI 与滑动窗口的成像亮度检测区域提取方法[J]. 公路交通技术,2026,42(1):224-234.ZHAO Jingxiang,LIAO Rong. Extraction method for imaging luminance detection regions based on dynamic ROI and sliding window[J]. Technology of Highway and Transport,2026,42(1):224-234.

[2]陈南，安然，张立平，等. 基于边缘点的动态 ROI 图像传输系统与 FPGA 实现[J]. 电子测量技术，2026，49(12)：250-257. DOI:10.19651/j.cnki.emt.2520182.

[3] szad670401. HyperLPR: High Performance Chinese License Plate Recognition Framework[CP/OL]. [2026-10-25]. https://github.com/szad670401/HyperLPR.

[4]IC 文艺青年. AXI stream 握手协议[EB/OL]. (2025-05-16)[2026-10-29]. https://zhuanlan.zhihu.com/p/1899109741222957775.

[5]XILINX. XUP High-Level Synthesis Design Flow[EB/OL]. [2026-10-31]. https://xilinx.github.io/xup_high_level_synthesis_design_flow/.

[6] ARM. AMBA 4 AXI4-Stream Protocol Specification: ARM IHI 0051A[S/OL]. 2010[2026-11-02].

[7]HUAMÁN A. Sobel Derivatives[EB/OL]. [2026-11-02].

[8]ROSENFELD A, PFALTZ J L. Sequential operations in digital picture processing[J]. Journal of the ACM, 1966, 13(4): 471-494.